
task-app 30日間カリキュラム

Day 23

週次レポートを表示しよう

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	3
今日のゴール	3
始める前の前提	4
なぜこれを作るのか	5
実装ステップ一覧	7
Step 0: 週次レポート API(getWeeklyReport)を自分で書く(読む目安: 14 分)	8
Step 1: プロジェクト統計の集計ロジック(読む目安: 5 分)	17
Step 2: 統計テーブルを表示(読む目安: 5 分)	21
Step 3: 週次レポートAPIの概要(読む目安: 3 分)	25
Step 4: 週次レポートページの基本構造(読む目安: 5 分)	28
Step 5: サマリーカードを表示(読む目安: 5 分)	37
Step 6: グラフを表示する(読む目安: 5 分)	41
Step 7: 動作確認(読む目安: 3 分)	52
完成コード全体	62
今日のまとめ	115
つまずきポイント	116
今日学んだ用語	117
理解チェック	118
追加課題: 2 週間の選択肢を追加する	119
次回予告	119
次に読むもの	120
奥付	120

前回の振り返り

Day 22 では Recharts ライブラリを使ってステータス別・優先度別の円グラフを実装しました。件数を数えるのはサーバーに任せ、ブラウザ側は `map` で日本語ラベルを足すだけにする分担も確認しました。`ResponsiveContainer` でのレスポンス対応も学んだので今日はプロジェクト別統計テーブルと週次レポート機能に取り組みます。

今日のゴール

レポートページのプロジェクト別統計テーブルを完成形と照合し、週次レポートページでグラフ付きの詳細レポートを表示します。テーブルで進捗を一覧表示し、折れ線グラフ・棒グラフで推移を可視化します。

この日はまずサーバー側の週次レポート API(`getWeeklyReport`)を自分で書きます。そのあと画面をつなぎます。

スクリーンショット: 今日の出発点になるレポートページです。Day 21 のカードと Day 22 の円グラフが並んでいます。Day 22 を終えた時点の画面なので今日の成果である週次レポートへのリンクはまだありません。

Day 23: 週次レポートを表示しよう



図1: レポートページ。統計カード4枚の下に、ステータス別タスクと優先度別タスクの円グラフが横に2枚並んでいる

始める前の前提

- Day 22 のグラフ表示が動いています
- 直近数週間のうちに、自分が担当のタスクを「完了」にした記録が1件以上あります
- `/report` を開いて統計カードとグラフを確認できます
- 週次レポートは集計結果の読み方も大事なので数字が少ない場合は練習用データを追加してから確認します
- 初期データの完了タスクは1件だけで、担当者も別の人
- `/task` で自分を担当者にしたタスクを2~3件作り、Day 16 の手順で「完了」にしてから開きます

なぜこれを作るのか

Day 21・22 では「今この瞬間」の数字を見てきました。でも「先週より進んだのか」は期間で区切って比べないと分かりません。プロジェクトごとの進捗を週単位でまとめ、変化を追えるようにします。

例え話: プロジェクト統計は「学校の通信簿」です。各教科(プロジェクト)ごとに成績(進捗率)や勉強時間(作業時間)が書かれています。通信簿を見ればどの教科が順調でどこを頑張るべきかが一目で分かります。

週次レポートの全体フロー

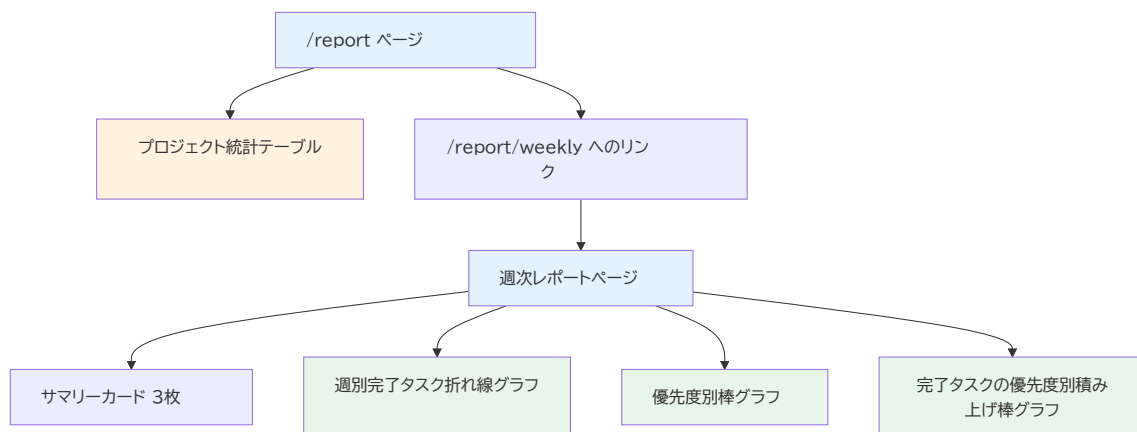


図 2: 週次レポートの全体フロー

図の左側の `/report` は Day 21・22 で作ったページです。今日はそこのプロジェクト統計テーブルを完成形と照合し、右へ渡るリンクを付けます。右側の `/report/weekly` は今日はじめて作るページで、サマリーカード 3 枚とグラフ 3 枚が並びます。

Day 23: 週次レポートを表示しよう

ページを2枚に分けるのは見たい時間の幅が違うためです。`/report` は「今の合計」を出すので期間を切りません。`/report/weekly` は「7日ずつの推移」を出すのでどこからどこまでを1週と数えるかを先に決める必要があります。その判断はすべて Step 0 で書くサーバー側の `getWeeklyReport` に置き、画面側は返ってきた配列を描くだけにします。集計の決まりが2か所に散らばると表とグラフで数字が食い違ったときに原因を追えなくなります。

やること / やらないこと

やること	やらないこと
プロジェクト別統計テーブル	<code>/report</code> 側で <code>task.getAll</code> を再集計する実装
週次レポートAPI呼び出し	ユーザー別フィルターUI
折れ線グラフで完了推移表示	カスタムテーブル作成
棒グラフで高・緊急の完了推移と優先度別内訳を表示	新規グラフライブラリ導入
	週次レポートの出力ページ (<code>/report/weekly/export</code>)

新しく学ぶ概念

概念	読み方	役割	例え
<code>projectStats</code>	—	プロジェクト別集計	通信簿の各教科
Table	テーブル	表形式の表示	Excel の表

概念	読み方	役割	例え
<code>getWeeklyReport</code>	—	週次データ取得API	週間天気予報
LineChart	ラインチャート	折れ線グラフ	気温の推移グラフ

開発サーバーは前の Day から動かしたまま使います。止めてあるときは

`npm run dev` で起動してから `http://localhost:3000` を開きます。

実装ステップ一覧

ステップ	作業内容	読む時間の目安
Step 0	週次レポート API (<code>getWeeklyReport</code>) を自分で書く	14 分
Step 1	プロジェクト統計の集計ロジック	5 分
Step 2	統計テーブルを表示	5 分
Step 3	週次レポートAPIの概要	3 分
Step 4	週次レポートページの基本構造	5 分
Step 5	サマリーカードを表示	5 分
Step 6	グラフを表示する	5 分
Step 7	動作確認	3 分

読む時間の合計(仮): 約45分です。

Day 23: 週次レポートを表示しよう

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 0: 週次レポート API(getWeeklyReport)を自分で書く(読む目安: 14分)

ゴール: Day 21 で作った `src/server/api/routers/report.ts` に `getWeeklyReport` を追記し、`api.report.getWeeklyReport` を自分で生やします。今回の追加は `reportRouter` の2本目の procedure です。新規ファイルではなく、前回作った `getOverview` の直後に足します。

週次レポートは「今の合計」ではなく「7日ごとの推移」を返します。だから Day 21 の `count` 中心の集計とは違い、今回は 期間を切る・週ごとに配列を作る・各週の中で `status` / `priority` を数える、という3段階になります。

最初に Day 21 の import 群を次の完成形へ置き換えます。今日から使う

`TRPCError`・`z`・`TASK_PRIORITY`・`USER_ROLE` が加わります。

```
// filepath: src/server/api/routers/report.ts(import 群の完成形)
import { TRPCError } from '@trpc/server';
import { z } from 'zod';
import { TASK_PRIORITY } from '@lib/constant/priority';
import { USER_ROLE } from '@lib/constant/roles';
import { TASK_STATUS } from '@lib/constant/status';
import { prisma } from '@lib/prisma';
import { createTRPCRouter, protectedProcedure } from
'../trpc';
```


`z` は入力検証、`TASK_PRIORITY` は優先度別の集計、`USER_ROLE` と `TRPCError` は他人のレポートを一般ユーザーから守る認可エラーに使います。

置き換えと書いたのは Day 21 では使わなかった 4 つが今日から加わるためです。取り込みを忘れたまま先へ進むと `z.object` を書いた行で「`z` が定義されていない」というエラーが出ます。エラーの文言はファイルの後ろのほうを指しますが直す場所はこの先頭の数行です。Day 21 で書いた `getOverview` の中身はそのまま残してください。今日はその下に足すだけで、既存の集計には触りません。

0-1. `getOverview` の直後に `input` を足す

```
// filepath: src/server/api/routers/report.ts(getOverview の直後に追加)

getWeeklyReport: protectedProcedure
  .input(
    z.object({
      weeks: z.number().int().min(1).max(12).default(4),
      userId: z.string().cuid().optional(),
    }).default({}),
  )
  .query(async ({ ctx, input }) => {
```

ここで受け取るのは `weeks` と `userId` の 2 つです。`weeks` は何週間分を見るかで、整数かつ最小 1、最大 12、未指定なら 4 です。`userId` は「誰の週次レポートを見るか」で、省略したときは自分自身のレポートになります。

Day 23: 週次レポートを表示しよう

0-2. 他人のレポートを見てよいかを確認する

```
// filepath: src/server/api/routers/report.ts(続き)
    if (input.userId && input.userId !==
ctx.session.userId) {
    if (ctx.session.role !== USER_ROLE.ADMIN) {
        throw new TRPCError({
            code: 'FORBIDDEN',
            message: '管理者権限が必要です',
        });
    }
}
```

ここが認可の入口です。`userId` を指定していてしかもそれが自分以外なら管理者だけに許可します。一般ユーザーが他人の週報を覗けてしまうと困るのでここで止めます。

0-3. 集計する期間を決める

```
// filepath: src/server/api/routers/report.ts(続き)

const targetUserId = input.userId ??
ctx.session.userId;

const now = new Date();

// 週バケットは「今日で終わる直近7日間」を最終週として7日刻みで遡る。

// 旧実装は最終週が「今日0時～現在」だけの進行中バケットで、「4週間」表示の

// 実力バー範囲が3週間+今日に縮んでいた。排他的上端を

// 明日0時に固定した完全な7日バケット×weeks本にすることで、ラベル・週平均の

// 分母と実際の集計範囲が一致し、範囲内タスクは必ずいずれかの週に入る。

// 日付ラベルは toISOString()(UTC)で出すため、バケット境界も UTC で刻む。

// ローカル時刻メソッドで刻むと、JST などのサーバーでラベルが1日ずれる。

const rangeEnd = new Date(now);
rangeEnd.setUTCHours(0, 0, 0, 0);
rangeEnd.setUTCDate(rangeEnd.getUTCDate() + 1);
```

rangeEnd を「明日の 00:00 UTC」に寄せているのが重要です。1週間を 7 日ぴったりで切るため最後の週も「今日 0 時から現在まで」ではなく、今日を含む 7 日間 に揃えます。

Day 23: 週次レポートを表示しよう

境界を UTC で刻む理由はこのあと週ラベルを `toISOString()` で作るからです。`toISOString()` は必ず UTC で日付を書き出します。もし境界だけ `setHours` のようなローカル時刻の書き方で刻むと日本時間で動くサーバーでは9時間ぶんずれ、ラベルの日付が1日前後します。刻む側と表示する側を UTC でそろえておけば手元の開発機と本番サーバーで時計の設定が違って同じ週区切りになります。

そのかわり、日本時間の深夜0時から朝9時までに完了したタスクはUTC ではまだ前日の扱いです。日本から見ると週の切れ目が朝9時に来ます。この教材では数時間のずれよりも「どの環境で動かしても数字が変わらない」ほうを選んでいきます。

```
// filepath: src/server/api/routers/report.ts(続き)
const startDate = new Date(rangeEnd);
startDate.setUTCDate(startDate.getUTCDate() -
input.weeks * 7);

const where = {
  status: TASK_STATUS.DONE,
  completedAt: { gte: startDate, lte: now },
  assigneeId: targetUserId,
  project: { members: { some: { userId:
targetUserId } } },
};
```

ここで期間の下端を作り、`where` に閉じ込めます。今回は「その期間に完了したタスク」だけを見るので `completedAt` を軸にしています。

この `where` は集計対象を絞っています。1つ目は `completedAt` が `null` の行、つまりまだ終わっていないタスクです。2つ目は `assigneeId` が対象ユーザーと違う行で、他人が片づけた仕事は混ざりません。3つ目は `startDate` より前の完了で、指定した週数の外にある過去は数えません。週次レポートは「この人がこの期間に終わらせた量」を出す画面なので残タスクの件数はここでは扱いません。さらに `status: TASK_STATUS.DONE` で現在も完了している行に限ります。

`members.some` は対象ユーザーの現在の参加状況を確認めます。担当者の名前が残っていてもプロジェクトから外れたあとは集計しません。

落とす条件を1か所へまとめておくところのあとの週ごとの絞り込みは日付の比較だけで済みます。もし週ごとの絞り込み側でも担当者の条件を書き直していたら片方を直し忘れた瞬間にグラフごとの数字が食い違います。

0-4. 期間内タスクを 1 回で取る

```
// filepath: src/server/api/routers/report.ts(続き)
const tasks = await prisma.task.findMany({
  where,
  select: {
    id: true,
    completedAt: true,
    status: true,
    priority: true,
  },
});
```

Day 23: 週次レポートを表示しよう

ここでは週ごとに使う材料を 1 回で取ります。`id` に加えて、実際に使う `completedAt` (どの週に入るか)と、あとで数える `status` と `priority` だけを `select` に並べます。プロジェクトへの参加条件は `where` で判定済みなので、集計後に使わないプロジェクト情報は取得しません。

週ごとに `findMany` を呼ぶ書き方もできますが4週間なら4回、12週間なら12回と問い合わせが増えていきます。期間全体を1回で取り、週への仕分けはこのあとの `filter` に任せると週数を増やしても問い合わせは1回のままです。読み込むタスクの件数は変わらないのに往復の回数だけが減ります。この考え方は今日の最後の「Pro パターンで書こう」でもう一度出てきます。

0-5. 1週ずつバケットを作る

```
// filepath: src/server/api/routers/report.ts(続き)
const weeklyData = Array.from({ length: input.weeks
}, (_, i) => {
  const weekStart = new Date(startDate);
  weekStart.setUTCDate(weekStart.getUTCDate() + i *
7);

  const weekEnd = new Date(weekStart);
  weekEnd.setUTCDate(weekEnd.getUTCDate() + 7);

  const weekTasks = tasks.filter(
    (task) => task.completedAt && task.completedAt
    >= weekStart && task.completedAt < weekEnd,
  );
```

`Array.from({ length: input.weeks }, (_, i) => ...)` は「必要な週数ぶんだけ箱を作る」書き方です。各週について `weekStart` と `weekEnd` を作り、その範囲に入るタスクだけを `filter` で抜き出します。終了側を `< weekEnd` にしているのと同じタスクが次の週と二重に数えられません。

```
// filepath: src/server/api/routers/report.ts(続き)
return {
  week: `${i + 1}週目`,
  weekStart: weekStart.toISOString().split('T')[
0],
  totalCompleted: weekTasks.length,
  byStatus: Object.fromEntries(
    Object.values(TASK_STATUS).map((status) => [
      status,
      weekTasks.filter((t) => t.status ===
status).length,
    ]),
  ),
},
```

`week` は画面表示用ラベル、`weekStart` はその週の開始日です。`byStatus` は `TASK_STATUS` を1つずつ回して件数を数え、`Object.fromEntries` で `{ TODO: 0, DONE: 5, ... }` の形へ戻しています。今回の取得対象は完了タスクだけなので、完了以外の件数は0です。進行中タスクの推移を表すデータではありません。

Day 23: 週次レポートを表示しよう

```
// filepath: src/server/api/routers/report.ts(続き)
    byPriority: Object.fromEntries(
      Object.values(TASK_PRIORITY).map((
priority) => [
      priority,
      weekTasks.filter((t) => t.priority ===
priority).length,
    ]),
    ),
  };
});
```

`byPriority` も考え方は同じです。Day 23 のグラフはこの `weeklyData` をクライアント側で `chartData` と `priorityData` に組み替えて使います。server側の役目は「週ごとの集計済み材料を返すところ」までです。

0-6. 最後に返して閉じる

```
// filepath: src/server/api/routers/report.ts(続き)
    return {
      weeks: input.weeks,
      startDate: startDate.toISOString(),
      endDate: now.toISOString(),
      weeklyData,
      totalCompleted: tasks.length,
    };
  }),
```


最後の `}}),` は `getWeeklyReport` 1本を閉じる行です。ルーター全体を閉じる `});` は Day 21 で書いた最終行がそのまま使えるのでここでは足しません。0-1 から 0-6 を貼る位置はその `});` の1行前です。

これで `reportRouter` は `getOverview` と `getWeeklyReport` の2本立てになりました。`root.ts` は Day 21 で `report: reportRouter` を登録済みなので今日は追加の登録作業は不要です。

確認ポイント:

- `src/server/api/routers/report.ts` の `getOverview` の直後に `getWeeklyReport` を追記できました
- `weeks` / `userId` の入力検証、管理者チェック、週バケット生成まで 完成版と同じ順序で書けました
- ファイルの最終行の `});` は Day 21 のまま1つで、増えていません
- `root.ts` は Day 21 時点の `report: reportRouter` のままでよいと理解できました
- `npx tsc --noEmit` で型エラーが出ていません

Step 1: プロジェクト統計の集計ロジック(読む目安: 5分)

ゴール: レポートページ(`/report`)に表示するプロジェクト統計の構造を理解します。完成版のコードでは Day 21 で導入した `api.report.getOverview` の `projectStats` をそのまま描画し、ここで再集計はしません。

Step 1・2 のコードは読み比べ用です。 `overview`、`projectStats`、`Table import`、「プロジェクト統計」カードを追加し直しません。

Day 23: 週次レポートを表示しよう

統計テーブルに表示する項目

項目	参照先	意味
プロジェクト	<code>stat.name</code>	プロジェクト名
タスク数	<code>stat.totalTasks</code>	タスク総数
完了	<code>stat.completedTasks</code>	完了タスク数
進捗	<code>stat.progress</code>	進捗率(%)
作業時間	<code>stat.totalTimeHours</code>	作業時間(h)

計算の流れ

手順	処理	例
1	<code>api.report.getOverview</code> を呼ぶ	server 側で全件集計
2	<code>overview.projectStats</code> を受け取る	A プロジェクトの集計行が入る
3	各行を <code>TableRow</code> に流し込む	進捗 30.0% と表示
4	<code>toFixed(1)</code> (小数第1位に丸める) で表示だけ整える	8.0h

```
// filepath: src/app/report/page.tsx
// Day 21 で追加済みの overview 取得
const { data: overview, isLoading } =
  api.report.getOverview.useQuery();
```

上記は Day 21 で追加済みのコードです。まだ追加していない場合は追加してください。

`useQuery` (データ取得のフック)はサーバーから届いた値を `data` に、取得中かどうかを `isLoading` に入れてくれます。画面はこの2つを見て表示を切り替えます。

確認ポイント:

- `overview` を取得できています
- 表の5項目が `projectStats` に入っていると理解しました

```
// filepath: src/app/report/page.tsx
// server 側で作られた projectStats をそのまま使う
const projectStats = overview?.projectStats ?? [];
```

`?? []` (左が無いとき空配列を使う書き方)を付けています。`overview` がまだ届いていない瞬間でも `projectStats` は空配列になり、後の `.map` がエラーになりません。

Day 23: 週次レポートを表示しよう

確認ポイント:

- クライアント側で再集計していません
- `projectStats` が配列として扱えます

```
{/* filepath: src/app/report/page.tsx */}  
{/* 描画時だけ小数第1位へ整える */}  
{projectStats.map((stat) => (  
  <TableRow key={stat.id}>  
    <TableCell className="font-medium">{  
stat.name}</TableCell>  
    <TableCell className="text-right">{  
stat.totalTasks}</TableCell>  
    <TableCell className="text-right">{  
stat.completedTasks}</TableCell>  
    <TableCell className="text-right">{  
stat.progress.toFixed(1)}%</TableCell>  
    <TableCell className="text-right">{  
stat.totalTimeHours.toFixed(1)}h</TableCell>  
  </TableRow>  
))}
```

確認ポイント:

- `progress` / `totalTimeHours` を表示時だけ整形しています
- `projectStats` の各要素がそのままテーブル行になります

`projectStats` 自体は `reportRouter.getOverview` の中で `groupBy` と `count` を使って作られています。 `/report` 側では集計の再実行は不要です。

Step 2: 統計テーブルを表示(読む目安: 5分)

ゴール: Table コンポーネントでプロジェクト統計を表形式で表示します。

Day 21 で表示済みのテーブルを確認します。以下のコードを2つ目のテーブルとして貼り付けないでください。

```
// filepath: src/app/report/page.tsx
// Table 関連のインポートを追加
import {
  Table, TableBody, TableCell,
  TableHead, TableHeader, TableRow,
} from '@component/ui/table';
```

shadcn/ui の表は1つの万能部品ではなく、`<table>` の各要素に対応する小さな部品の集まりです。だから6つをまとめて取り込みます。1つでも書き漏らすとその名前が定義されていないというエラーで画面が真っ白になります。どの部品がHTMLのどのタグに当たるかはすぐ下の表にまとめてあります。

確認ポイント:

- Table 関連の6つのコンポーネントをインポートしました

Table コンポーネントの構造

コンポーネント	役割	HTML相当
Table	テーブル全体	<code><table></code>
TableHeader	ヘッダー領域	<code><thead></code>
TableHead	見出しセル	<code><th></code>
TableBody	データ領域	<code><tbody></code>
TableRow	行	<code><tr></code>
TableCell	データセル	<code><td></code>

```

{/* filepath: 読み比べ用サンプル(実ファイルには対応しませ
ん) */}
{/* テーブルのヘッダー定義 */}
<Card>
  <CardHeader>
    <CardTitle>プロジェクト統計</CardTitle>
  </CardHeader>
  <CardContent>
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead className="w-[200px]">
            プロジェクト</TableHead>
          <TableHead className="text-right">
            タスク数</TableHead>
          <TableHead className="text-right">
            完了</TableHead>
          <TableHead className="text-right">
            進捗</TableHead>
          <TableHead className="text-right">
            作業時間</TableHead>
        </TableRow>
      </TableHeader>

```

確認ポイント:

- `TableHeader` の中に `TableRow` と `TableHead` があります
- ヘッダー5列を定義しました

Day 23: 週次レポートを表示しよう

`TableHeader` の中に見出し行の `TableRow` を置き、その中へ見出しセルの `TableHead` を並べます。この入れ子はブラウザに「ここが表の見出し行」と伝えるための形です。

```
{/* filepath: 読み比べ用サンプル(実ファイルには対応しません) */}
{/* テーブル本体(mapで各行を生成) */}
<TableBody>
  {projectStats?.map((stat) => (
    <TableRow key={stat.id}>
      <TableCell className="font-medium">
        {stat.name}</TableCell>
      <TableCell className="text-right">
        {stat.totalTasks}</TableCell>
      <TableCell className="text-right">
        {stat.completedTasks}</TableCell>
      <TableCell className="text-right">
        {stat.progress.toFixed(1)}%</TableCell>
      <TableCell className="text-right">
        {stat.totalTimeHours.toFixed(1)}h</TableCell>
    </TableRow>
  ))}
</TableBody>
```

確認ポイント:

- テーブルにプロジェクト名が並びます

- 数値が `text-right` で右寄せ表示されます

shadcn/ui の Table はHTML のテーブル要素をラップしたものです。

`text-right` で数値を右寄せにすると表が見やすくなります。

スクリーンショット: 完成形と照合するテーブルです。

プロジェクト統計				
プロジェクト	タスク数	完了	進捗	作業時間
Webサイトリニューアル	6	4	66.7%	33.3h

図3: レポートページ。赤枠の中はプロジェクト統計テーブルで、プロジェクト名・タスク数・完了数・進捗率・作業時間が並ぶ

Step 3: 週次レポートAPIの概要(読む目安: 3分)

ゴール: 週次レポートAPIのパラメータとレスポンス構造を理解します。このステップはコードを読んで理解するだけです。

```
// filepath: src/app/report/weekly/page.tsx(Step 4 で作成)
// 週次レポートAPIの呼び出しイメージ(クライアント側で呼ぶ)
api.report.getWeeklyReport.useQuery({
  weeks: 4,
});
```

Day 23: 週次レポートを表示しよう

この1行がStep 0 で書いたサーバー側の処理を呼び出す入口です。`weeks: 4` を渡すとサーバーは直近4週間ぶんの集計を組み立てて返します。Step 0 で `weeks` に `.min(1).max(12)` を付けたので13 を渡した時点で tRPC が入力を弾き、データベースまでは届きません。画面側で範囲を確かめる `if` を書かなくてよいのはこの検証をサーバーに持たせたからです。

返ってくる値の形は下の3つの表のとおりです。Step 6 のグラフはこのうち `weeklyData` だけを組み替えて使います。ここで形を頭に入れておくとStep 4以降でどのプロパティに触っているのか迷わずに済みます。

APIのパラメータ

パラメータ	型	必須	説明
weeks	number	いいえ(デフォルト: 4)	取得する週数(1~12)
userId	string	いいえ	特定ユーザーに絞る

APIのレスポンス

プロパティ	型	説明
weeks	number	指定した週数
startDate	string	集計開始日
endDate	string	集計終了日
weeklyData	array	週ごとのデータ配列
totalCompleted	number	期間内の完了総数

weeklyData の各要素

プロパティ	型	説明
week	string	1週目 のような週ラベル
weekStart	string	その週の開始日 (YYYY-MM-DD)
totalCompleted	number	その週の完了数
byStatus	Record<string, number> (キーが文字 列・値が数値のオブジェク ト型)	完了タスクのステータス 別の件数
byPriority	Record<string, number>	優先度別の件数

サーバー側で Prisma を使って **completedAt** の日付範囲でタスクをフィルターし、週ごとに集計しています。

確認ポイント:

- APIのパラメータとレスポンスの構造を理解しました
- **weeklyData** が週ごとのデータ配列であることを把握しました
- レスポンスに **byStatus** と **byPriority** があり、今日の画面では **byPriority** から2種類の棒グラフを作ること理解了しました

Step 4: 週次レポートページの基本構造(読む目安: 5分)

ゴール: `/report/weekly` ページを作成し、API呼び出しと週数選択 UI を実装します。

```
// filepath: src/app/report/weekly/page.tsx
// インポート(日付・React・UIコンポーネント)
'use client';
import { formatDateOnly } from '@lib/date';
import { useState } from 'react';
import { AppLayout }
  from '@component/layout/app-layout';
import {
  Card, CardContent,
  CardHeader, CardTitle,
} from '@component/ui/card';
import { PageLoadingSpinner }
  from '@component/ui/loading-spinner';
```

確認ポイント:

- `formatDateOnly` をインポートしました
- `PageLoadingSpinner` のパスが `@/component/ui/loading-spinner` です

先頭の `'use client'` (ブラウザ側で動く宣言)を書くとこのページはブラウザ側で動く画面になります。 `useState` などブラウザ側で動く機能を使うためこの宣言が必要です。

```
// filepath: src/app/report/weekly/page.tsx
// インポート(Select・Recharts・定数)
import {
  Select, SelectContent,
  SelectItem, SelectTrigger,
  SelectValue,
} from '@component/ui/select';
import {
  Bar, BarChart, CartesianGrid,
  Legend, Line, LineChart,
  ResponsiveContainer,
  Tooltip, XAxis, YAxis,
} from 'recharts';
import {
  TASK_PRIORITY, TASK_PRIORITY_COLORS,
} from '@lib/constant/priority';
import { api } from '@trpc/react';
```

確認ポイント:

- Recharts の10 種類のコンポーネントをインポートしました
- `TASK_PRIORITY_COLORS` をインポートしました

今日初めて使う Recharts の部品を先に紹介します。 `CartesianGrid` はグラフ背景の目盛り線を引きます。 `XAxis` / `YAxis` は横軸と縦軸を描きます。 `Line` は折れ線グラフの線1本、 `Bar` は棒グラフの1系列です。どれも Step 6 で実際に配置します。

Day 23: 週次レポートを表示しよう

```
// filepath: src/app/report/weekly/page.tsx
// API呼び出しとローディング処理
const CHART_PRIMARY_COLOR = '#8884d8';

export default function WeeklyReportPage() {
  const [weeks, setWeeks] = useState('4');

  const {
    data: reportData,
    isLoading,
  } = api.report.getWeeklyReport.useQuery({
    weeks: Number.parseInt(weeks, 10),
  });

  if (isLoading) {
    return <PageLoadingSpinner />;
  }
}
```

初期値を数字の `4` ではなく文字列の `'4'` にしているのはあとで置く `Select` が文字列でしか値をやり取りしないからです。ところが `getWeeklyReport` の `weeks` は数値なのでそのまま渡すと型が合いません。

`Number.parseInt(weeks, 10)` はその橋渡しで、第2引数の `10` は「10進数として読む」という指定です。

`isLoading` のうちに `return` で打ち切るのも大事です。この後に書くコードは `reportData?.totalCompleted ?? 0` のように `?.` と `?? 0` で守ってあるのでここを飛ばしても画面がエラーで止まるわけではありません。代わりに、まだ何も届いていない状態のまま数字の `0` と空っぽのグラフが一瞬だけ描かれます。読み手

にはそれが「この期間は0件だった」という結果に見えてしまいます。まだ届いていないことはスピナーではっきり伝えます。Day 09 のプロジェクト一覧でも、`projectsLoading` が真の間はスピナーを返して先へ進ませませんでした。

確認ポイント:

- `useState('4')` で初期値4週間を設定しています
- `isLoading` のときスピナーを表示しています

Day 23: 週次レポートを表示しよう

```
{/* filepath: src/app/report/weekly/page.tsx */}
{/* 週数選択のSelectコンポーネント */}
<div className="w-[150px]">
  <Select
    value={weeks}
    onChange={setWeeks}>
    <SelectTrigger>
      <SelectValue placeholder="期間" />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="4">
        4週間
      </SelectItem>
      <SelectItem value="8">
        8週間
      </SelectItem>
      <SelectItem value="12">
        12週間
      </SelectItem>
    </SelectContent>
  </Select>
</div>
```

確認ポイント:

- 4・8・12週間の選択肢があります
- `onChange` で `setWeeks` を呼んでいます

`useState` で週数を管理します。ユーザーが週数を変更すると `useQuery` が自動的に再取得します。

ページ全体のJSX構造

階層	要素	役割
1	<code><AppLayout></code>	共通レイアウト
2	<code><div className="space-y-6"></code>	縦方向の余白
3	ヘッダー(h1 + Select)	タイトルと期間選択
3	<code>grid grid-cols-1 md:grid-cols-3</code>	3枚のサマリーカード
3	<code>grid grid-cols-1 lg:grid-cols-2</code>	グラフ3枚

ページの骨格を完成させる

上の表を実際のコードにすると次の骨格になります。 `if (isLoading)` の直後に、この `return` を書きます。

Day 23: 週次レポートを表示しよう

```
// filepath: src/app/report/weekly/page.tsx
// ページの骨格(return から関数の閉じ括弧まで)
return (
  <AppLayout>
    <div className="space-y-6">
      <div className="flex items-center
        justify-between">
        <h1 className="text-3xl font-bold">
          週次レポート
        </h1>
        {/* 週数選択の Select を置く */}
      </div>
      {/* Step 5: サマリーカード3枚 */}
      {/* Step 6: グラフ3枚のグリッド */}
    </div>
  </AppLayout>
);
}
```

最後の `}` は `WeeklyReportPage` 関数を閉じる括弧です。この Step の前半で書いた週数選択の `Select` は `src/app/report/weekly/page.tsx` の `h1` の隣にあるコメントの行と置き換えます。Step 5 と Step 6 で作るカードとグラフも、対応するコメントの行と置き換えていきます。

確認ポイント:

- `return` の一番外側が `<AppLayout>` になっています
- 関数を閉じる `}` まで書けています

スクリーンショット: 読み込んでいる間はスピナーだけが出ます。



図 4: 読み込み中の週次レポート。画面の中央に丸いスピナーだけが出ている

週次ページを直接 URL 入力しなくても開けるよう、

`src/app/report/page.tsx` のヘッダーへリンクを追加します。既存 import に次を追加してください。

```
// filepath: src/app/report/page.tsx
import { ArrowRight } from 'lucide-react';
import Link from 'next/link';
```

レポート見出しの右側へリンクを置きます。並べるには囲みが要ります。まず見出しと説明文を `<div>` で1つにまとめてください。その外側を

```
flex flex-col gap-4 items-start sm:flex-row sm:items-center sm:justify-between
```

を付けた `<div>` で包みます。この囲みが無いとリンクは見出しの下へ落ちます。次のコードはその囲みの中、見出しをまとめた `<div>` の後ろへ貼ります。`Link` は Next.js が用意したページ移動用の部品で、`<a>` タグと違って移動先の中身だけを

Day 23: 週次レポートを表示しよう

差し替えます。だからサイドバーやヘッダーは表示されたまま残り、画面が一度白くなってから描き直される動きになりません。`ArrowRight` は右向き矢印のアイコンで、押した先へ進むことを文字より速く伝えます。

```
{/* filepath: src/app/report/page.tsx */}
<Link
  href="/report/weekly"
  className="inline-flex items-center gap-2
    text-sm font-medium text-primary
    hover:underline"
>
  週次レポートを見る
  <ArrowRight className="h-4 w-4" />
</Link>
```

`href` に書いた `/report/weekly` は Step 4 で作ったファイルの置き場所とそろえてあります。`src/app/report/weekly/page.tsx` という場所がそのまま URL になるのが Next.js のページの決まりで、綴りが1文字でも違うと 404 ページに飛びます。

`inline-flex items-center gap-2` は文字と矢印を横に並べ、縦位置を中央でそろえ、その間に隙間を空ける指定です。この3つを外すと矢印が文字の下端にずれて落ちたように見えます。

確認ポイント:

- `/report` から週次レポートを開けます
- 「プロジェクト統計」テーブルは1つだけ表示されます

Step 5: サマリーカードを表示(読む目安: 5分)

ゴール: 完了タスク合計・週平均・対象期間の3枚のカードを表示します。

```
{/* filepath: src/app/report/weekly/page.tsx */}
{/* 完了タスク合計カード */}
<div className="grid grid-cols-1
  md:grid-cols-3 gap-4">
  <Card>
    <CardContent className="pt-6">
      <p className="text-sm
        text-muted-foreground mb-1">
        完了タスク合計
      </p>
      <p className="text-3xl font-bold">
        {reportData?.totalCompleted ?? 0}
      </p>
    </CardContent>
  </Card>
```

外枠に `grid-cols-1 md:grid-cols-3` と2つ書いてあるのは画面幅で並べ方を切り替えるためです。スマホでは縦に1枚ずつ、`md` (横幅 768px 以上)から横3列になります。3列のまま縮めると `text-3xl` の数字が折り返して読めなくなります。

Day 23: 週次レポートを表示しよう

`{reportData?.totalCompleted ?? 0}` の `?? 0` が何をしているかは3つの状態に分けると見えてきます。1つ目は読み込み中で、`isLoading` が真の間はスピナーを返すためこの行までは来ません。2つ目は通信が失敗したときで、`reportData` は `undefined` のまま残ります。3つ目は通信に成功して完了タスクが1件も無かったときです。0 と表示してよいのは3つ目だけです。

`?? 0` は失敗を見分けません。2つ目でも `0` と出るので読者には「今週は完了0件」との区別が付きません。`useQuery` は取得に失敗した理由を `error` にも入れて返すので失敗は本来そちらを見て「読み込みに失敗しました」と別に伝えます。この画面ではまだ `error` を受け取っていないため失敗を伝える表示がありません。`?? 0` がしているのは数字の場所が空欄になるのを防ぐことです。

学習段階と現在の完成版の違い: この Day 23 で作る週次ページは、週数を画面内の state だけで切り替える学習段階の実装です。現在の完成版には、通信エラーを本当の0件と分ける表示、選んだ `weeks` と URL パラメータの同期、CSV(カンマで項目を区切り、表計算ソフトで開けるファイル)出力へのリンクが加わっています。これらは現在の完成版で追加された機能なので、この Step ではコードを足しません。

確認ポイント:

- `grid-cols-3` で3列レイアウトになっています
- 完了タスク合計の数値が表示されます

```

{/* filepath: src/app/report/weekly/page.tsx */}
{/* 週平均カード */}
<Card>
  <CardContent className="pt-6">
    <p className="text-sm
      text-muted-foreground mb-1">
      週平均
    </p>
    <p className="text-3xl font-bold">
      {reportData?.totalCompleted
        ? Math.round(
            reportData.totalCompleted
              / Number.parseInt(weeks, 10)
          )
        : 0}
    </p>
  </CardContent>
</Card>

```

週平均は完了タスクの合計を選択した週数で割ります。API へ送る週数も、分母も同じ `weeks` から作るので一致します。期間は今日を含む7日間を最後の週として区切りますが今日の残り時間はまだ集計できません。カードはその時点までに完了した件数を選択週数で割った値です。

`Number.parseInt(weeks, 10)` が 0 になることはありません。`Select` が返す値は `4` `8` `12` の3つだけなので0で割ってしまう心配は要らないという理屈です。

Day 23: 週次レポートを表示しよう

確認ポイント:

- `Math.round` で小数を丸めています
- `Number.parseInt` で文字列の `weeks` を数値に変換しています

```
{/* filepath: src/app/report/weekly/page.tsx */}
{/* 対象期間カード(UTC の日付を表示) */}
<Card>
  <CardContent className="pt-6">
    <p className="text-sm
      text-muted-foreground mb-1">
      集計期間(UTC基準)</p>
    <p className="text-lg font-semibold">
      {reportData?.startDate
        && reportData?.endDate
        ? `${formatDateOnly(reportData.startDate)} - ${
formatDateOnly(reportData.endDate)}`
        : '-'}
    </p>
  </CardContent>
</Card>
</div>
```

確認ポイント:

- `formatDateOnly` で UTC の日付を表示しています
- データがないときは `'-'` を表示しています

3枚のカードは1つの `reportData` から値を取り出し、合計・週平均・対象期間という3種類の見せ方にしています。対象期間は UTC で集計した日付です。`formatDateOnly` は日時文字列の日付部分を使うためブラウザのタイムゾーンで前日や翌日にずれません。見出しにも「UTC」を付け、集計の基準を伝えます。

週次レポートの表示項目

カード	表示内容	計算方法
完了タスク合計	期間内の完了数	API が返す値
週平均	週あたり平均	完了数 / 週数
対象期間	集計期間	開始日 - 終了日

Step 6: グラフを表示する(読む目安: 5分)

ゴール: Recharts で折れ線グラフと棒グラフを表示して週次推移を可視化します。

実装: Step 4 で書いた `if (isLoading)` の下、`return` 文の前に追加してください。`return` の中に書くと JSX の途中に `const` を置くことになって構文エラーになります。

Day 23: 週次レポートを表示しよう

```
// filepath: src/app/report/weekly/page.tsx
// グラフ用データの変換処理(完了数・優先度)
const chartData =
  reportData?.weeklyData.map((week) => ({
    name: week.week,
    completed: week.totalCompleted,
    high:
      week.byPriority[TASK_PRIORITY.HIGH]
      ?? 0,
    urgent:
      week.byPriority[TASK_PRIORITY.URGENT]
      ?? 0,
  }));
```

確認ポイント:

- `chartData` は完了数と優先度データを持っています

Recharts のグラフは1週分を1オブジェクトにまとめ、各系列の値をキーに持つ配列を受け取ります。 `weeklyData` はこの形と違うので `name` や `completed` をキーに持つ形へ組み替えています。

実装: `chartData` のすぐ下、`return` 文の前に続けて書きます。

```
// filepath: src/app/report/weekly/page.tsx
// グラフ用データの変換処理(完了タスクの優先度別)
const priorityData = reportData?.weeklyData.map((week) =>
({
  name: week.week,
  low: week.byPriority[TASK_PRIORITY.LOW] ?? 0,
  medium: week.byPriority[TASK_PRIORITY.MEDIUM] ?? 0,
  high: week.byPriority[TASK_PRIORITY.HIGH] ?? 0,
  urgent: week.byPriority[TASK_PRIORITY.URGENT] ?? 0,
})));
```

ここで数えている材料は `completedAt` が期間内に入っているタスクだけです。
Step 0 の `where` が `completedAt` を軸に絞っているのもまだ終わっていない
タスクは1件も混ざりません。担当が自分以外の行も、期間より古い行も、同じ
`where` で落ちています。

Day 23: 週次レポートを表示しよう

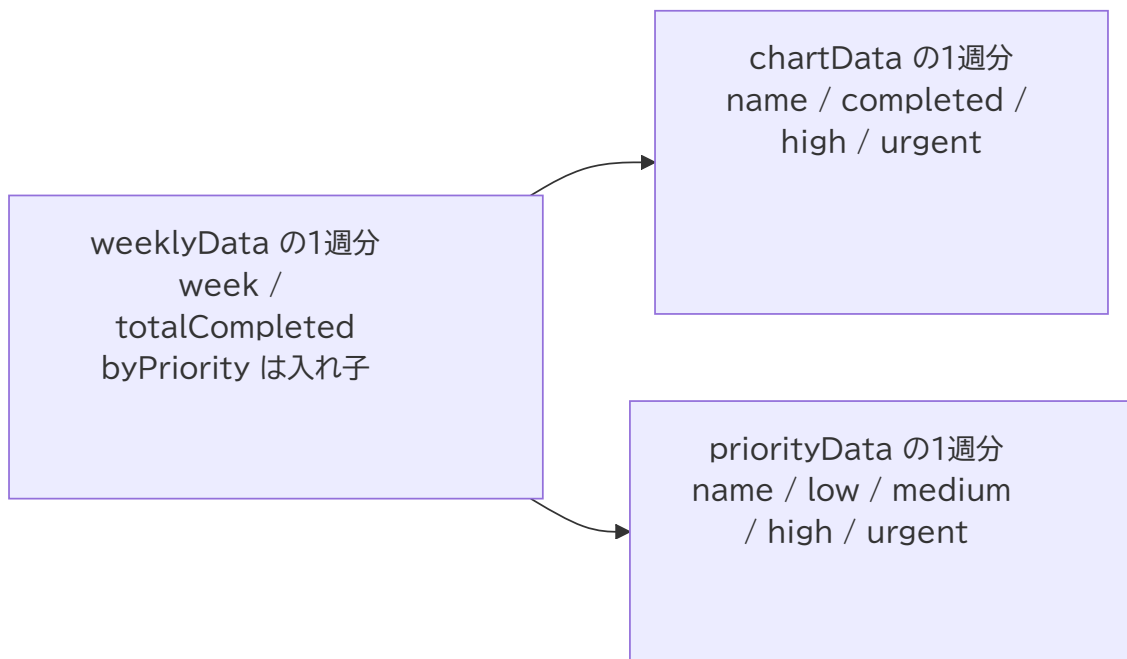


図5: Step 6: グラフを表示する

左は入れ子、右は入れ子のないひと並びです。Recharts に渡すときは右の形がいちばん素直なので同じ材料から2種類のひと並びを作ります。グラフの棒が出ないときは `dataKey` に書いた名前が右側の列名と合っているかを見てください。

積み上げ棒では完了タスクを優先度で分けます。まだ進行中の仕事は集計に含まれません。完了から戻したタスクも Day 15 の更新処理で完了日時が `null` になり、集計から外れます。

低・中・高・緊急の4系列を積み上げると棒の高さがその週の完了数になります。折れ線グラフと合計が一致するか確認してみましょう。

確認ポイント:

- `priorityData` は完了タスクの優先度別データを持っています

実装: Step 4 の骨格に置いた `{/* Step 6: グラフ3枚のグリッド */}` の行を、次の外枠と置き換えます。3枚のカードはこの `<div>` の中に入ります。

```
{/* filepath: src/app/report/weekly/page.tsx */}
{/* グラフ3枚を包む外枠 */}
<div className="grid grid-cols-1
  lg:grid-cols-2 gap-6">
```

開いたままの `<div>` を先に置きます。閉じタグは3枚目のカードの後ろに書くのでここで保存すると閉じタグ不足のエラーが出ます。続きを書けば消えます。

外枠を `grid` にしておくとその中に置いたカードが横に並びます。外枠が無いままカードだけを書くと `col-span` の指定は効く相手がいないので無視されます。その結果、グラフ3枚が縦へ積まれます。エラーは出ないので並び方だけが完成形と違う状態に気づけません。`grid-cols-1 lg:grid-cols-2` はスマートフォンの幅では1列、`lg` (横幅 1024px 以上)から2列という意味です。

Day 23: 週次レポートを表示しよう

```
{/* filepath: src/app/report/weekly/page.tsx */}
{/* 週別完了タスク数の折れ線グラフ */}

<Card className="col-span-1 lg:col-span-2">
  <CardHeader>
    <CardTitle>週別完了タスク数</CardTitle>
  </CardHeader>
  <CardContent>
    <div className="h-[300px]">
      <ResponsiveContainer width="100%"
        height="100%">
        <LineChart data={chartData ?? []}>
          <CartesianGrid
            strokeDasharray="3 3" />
          <XAxis dataKey="name" />
          <YAxis /><Tooltip /><Legend />
          <Line type="monotone"
            dataKey="completed"
            stroke={CHART_PRIMARY_COLOR}
            name="完了数" strokeWidth={2} />
        </LineChart>
      </ResponsiveContainer>
    </div>
  </CardContent>
</Card>
```

折れ線を選んだのはこのカードで見せたいものが件数そのものではなく増減の向きだからです。棒でも件数は読めますが週から週への向きは線のほうが速く目に入ります。だから `col-span-1 lg:col-span-2` で横幅を2つぶん取り、時間の流れを長く見せています。

`<div className="h-[300px]">` で高さを固定しているのは `ResponsiveContainer` に `height="100%"` を指定しているためです。親に高さが無いと 100% は 0 と計算され、グラフは1本も描かれません。Day 22 の円グラフでも、この高さ指定と `ResponsiveContainer` を組にして使いました。

`type="monotone"` は点と点を緩い曲線でつなぐ指定です。`chartData` が空配列でも `LineChart` は軸だけを描いて止まらないので完了が1件も無い週があっても画面は壊れません。

確認ポイント:

- `LineChart` で折れ線グラフを描画しています
- `col-span-1 lg:col-span-2` で、画面が広いとき横幅いっぱいに表示されます

Day 23: 週次レポートを表示しよう

```
{/* filepath: src/app/report/weekly/page.tsx */}
{/* 高・緊急の完了タスク数の棒グラフ */}

<Card>
  <CardHeader>
    <CardTitle>高・緊急の完了タスク数</CardTitle>
  </CardHeader>
  <CardContent>
    <div className="h-[300px]">
      <ResponsiveContainer width="100%"
        height="100%">
        <BarChart data={chartData ?? []}>
          <CartesianGrid strokeDasharray="3 3" />
          <XAxis dataKey="name" />
          <YAxis /><Tooltip /><Legend />
          <Bar dataKey="urgent" name="緊急"
            fill={TASK_PRIORITY_COLORS.URGENT} />
          <Bar dataKey="high" name="高"
            fill={TASK_PRIORITY_COLORS.HIGH} />
        </BarChart>
      </ResponsiveContainer>
    </div>
  </CardContent>
</Card>
```


`Bar` を2本しか置いていないのは `chartData` に `urgent` と `high` しか入れなかったからです。週次レポートで確かめたいのは「急ぎの仕事をどれだけ片づけたか」なので低と中の件数は落としてあります。優先度4段階すべての内訳は次の積み上げ棒グラフで見られます。数を全部出さない判断も、グラフを読みやすくするための仕事です。

`fill` に `TASK_PRIORITY_COLORS` を渡すと Day 22 の円グラフと同じ色で棒が塗られます。色の文字列を直接書いてしまうとあとで優先度の色を変えたときにこのグラフだけ古い色で取り残されます。

確認ポイント:

- `BarChart` で棒グラフを描画しています
- `TASK_PRIORITY_COLORS` で色分けしています

Day 23: 週次レポートを表示しよう

```
{/* filepath: src/app/report/weekly/page.tsx */}
{/* 完了タスクの優先度別積み上げ棒グラフのCard部分 */}
<Card>
  <CardHeader>
    <CardTitle>完了タスクの優先度別内訳</CardTitle>
  </CardHeader>
  <CardContent>
    <div className="h-[300px]">
      <ResponsiveContainer width="100%"
        height="100%">
        <BarChart data={priorityData ?? []}>
          <CartesianGrid strokeDasharray="3 3" />
          <XAxis dataKey="name" />
          <YAxis /><Tooltip /><Legend />
```

外側の作りは優先度グラフとそろえてありますが `data` に渡すのが `chartData` から `priorityData` へ変わっています。`XAxis` の `dataKey="name"` を書き換えずに済むのは2つの配列のどちらも `name` に「1週目」のような週ラベルを入れてあるからです。Step 6 の冒頭で形をそろえておいた効き目がここで出ます。

このコードブロックはカードの前半だけで、`</BarChart>` から先の閉じ括弧は次のブロックにあります。途中で保存するとエラー表示が出ますが続きを書けば消えるので手を止めないでください。

確認ポイント:

- `priorityData` を `BarChart` に渡しています

```

{/* filepath: src/app/report/weekly/page.tsx */}
{/* 4つのBarで積み上げ表示 */}
    <Bar dataKey="low" stackId="priority"
name="低"
        fill={TASK_PRIORITY_COLORS.LOW} />
    <Bar dataKey="medium" stackId="priority"
name="中"
        fill={TASK_PRIORITY_COLORS.MEDIUM} />
    <Bar dataKey="high" stackId="priority"
name="高"
        fill={TASK_PRIORITY_COLORS.HIGH} />
    <Bar dataKey="urgent" stackId="priority"
name="緊急"
        fill={TASK_PRIORITY_COLORS.URGENT} />
  </BarChart>
</ResponsiveContainer>
</div>
</CardContent>
</Card>
</div>

```

確認ポイント:

- `stackId="priority"` で積み上げ棒グラフになっています
- 4つの優先度が色分けで表示されます
- 3枚のグラフが `lg` 幅で2列に並び、折れ線グラフだけが横2つぶんを占めます

Day 23: 週次レポートを表示しよう

`stackId` は今日初登場の指定です。同じ `stackId` を持つ `Bar` 同士は横に並ばず1本の棒として積み上がります。

Day 22 で学んだ Recharts を週次レポートでも活用しています。

`LineChart` は推移の把握に、`BarChart` は比較に適しています。

スクリーンショット: 3つのグラフが並びます。



図6: 週次レポート。赤枠の中に、週別完了タスク数の折れ線グラフ、高・緊急の完了タスク数と完了タスクの優先度別内訳の棒グラフが並んでいる

Step 7: 動作確認(読む目安: 3分)

ゴール: 全体の表示を確認します。

開発サーバーが動いていればそのまま使います。止めてあるときだけ次のコマンドで起動します。

```
# filepath: ターミナル  
# 停止している場合だけ起動する  
npm run dev
```

確認ポイント:

- 開発サーバーが正常に起動しました
1. `/report` にアクセス
 2. 統計カード(Day 21)が表示されます
 3. 円グラフ(Day 22)が表示されます
 4. プロジェクト統計テーブルが表示されます
 5. 各プロジェクトの進捗率が正しい
 6. `/report/weekly` にアクセス
 7. 3枚のサマリーカードが表示されます
 8. 折れ線グラフが表示されます
 9. 「高・緊急の完了タスク数」と「完了タスクの優先度別内訳」の棒グラフが表示されます

スクリーンショット: 週次レポートページの全体です。

Day 23: 週次レポートを表示しよう



図 7: 週次レポートページ。サマリーカードの下に折れ線グラフ、その下に2つの棒グラフが並んでいる。横幅が1024px未満では棒グラフも縦に並ぶ

グラフの数字は前提の手順で自分が完了にしたタスクのぶんです。件数が違ってても実装の誤りではありません。写真では「高・緊急の完了タスク数」だけが空になっています。この棒グラフは **high** と **urgent** の2系列しか描かず、タスク作成ダイアログの既定は「中」だからです。優先度を「高」か「緊急」にしたタスクを完了にするとここにも棒が立ちます。

Pro パターンで書こう(週次レポートのデータ取得は Prisma の select でまとめる)

`select` をネストするとタスクとプロジェクトを1回のPrisma呼び出しで取得でき、N+1問題(一覧を1回取得したあと、要素ごとに追加のクエリを発行してしまう問題)を回避できます。なぜ直前の1文の書き方をするのか、**Before/After** で見比べてみましょう。

Before(改善前のコード)

```
// filepath: 読み比べ用サンプル(実ファイルには対応しません)
import { TASK_STATUS } from '@lib/constant/status';
import { prisma } from '@lib/prisma';

type WeeklyReportTask = {
  id: string;
  completedAt: Date | null;
  status: string;
  priority: string;
  project: {
    id: string;
    name: string;
  } | null;
};
```

取得条件は前後で共通です。対象ユーザーの認可は呼び出し元が担当します。この関数は、対象ユーザーが現在も参加しているプロジェクトの完了タスクを取得します。

Day 23: 週次レポートを表示しよう

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

export async function fetchWeeklyReportTasks(
  targetUserId: string,
  startDate: Date,
  endDate: Date,
): Promise<WeeklyReportTask[]> {
  const tasks = await prisma.task.findMany({
    where: {
      assigneeId: targetUserId,
      status: TASK_STATUS.DONE,
      project: { members: { some: { userId: targetUserId
    } } },
      completedAt: { gte: startDate, lt: endDate },
    },
  },
```

`gte` / `lt` (以上／未満のPrisma条件)で、`completedAt` が指定した期間内のタスクだけを絞り込みます。終了側を `lt` (未満)にするのは `endDate` ちょうど瞬間を次の週に含めるためです。週の境界を「開始以上・終了未満」でそろえると同じタスクが2つの週に二重で数えられません。

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。


```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

select: {
  id: true,
  completedAt: true,
  status: true,
  priority: true,
  projectId: true,
},
});

return await Promise.all(
  tasks.map(async (task) => {
    const project = await prisma.project.findUnique({
      where: { id: task.projectId },
      select: {
        id: true,
        name: true,
      },
    });
  });

return {
  id: task.id,
  completedAt: task.completedAt,
  status: task.status,
  priority: task.priority,
```

Day 23: 週次レポートを表示しよう

分かれ道は `select` に `projectId` しか入れていない点です。プロジェクト名が手元に無いのでそのあとの `tasks.map` で1件ずつ

`prisma.project.findUnique` を呼び直すしかなくなります。タスクが 30 件あれば最初の1回に加えて問い合わせが 30 回増えます。`Promise.all` で同時に走らせても回数そのものは減りません。

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

    project,
  };
},
);
}
```

このコードの問題点:

- タスクが 30 件あれば最初の取得 1 回に加えてプロジェクト取得が 30 回走ります
- `Promise.all` を使っていても DB への問い合わせ回数が増える構造は変わりません
- 週次レポートに担当者やプロジェクト情報を増やすたび、同じ N+1 が別の relation でも起きやすい

After(プロが書くコード)

```
// filepath: 読み比べ用サンプル(実ファイルには対応しません)
import { TASK_STATUS } from '@lib/constant/status';
import { prisma } from '@lib/prisma';

type WeeklyReportTask = {
  id: string;
  completedAt: Date | null;
  status: string;
  priority: string;
  project: {
    id: string;
    name: string;
  } | null;
};
```

取得条件は前後で共通です。対象ユーザーの認可は呼び出し元が担当します。この関数は、対象ユーザーが現在も参加しているプロジェクトの完了タスクを取得します。

Day 23: 週次レポートを表示しよう

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

export async function fetchWeeklyReportTasks(
  targetUserId: string,
  startDate: Date,
  endDate: Date,
): Promise<WeeklyReportTask[]> {
  return await prisma.task.findMany({
    where: {
      assigneeId: targetUserId,
      status: TASK_STATUS.DONE,
      project: { members: { some: { userId: targetUserId
    } } },
      completedAt: { gte: startDate, lt: endDate },
    },
  },
```

先頭が `return await prisma.task.findMany({` になっているのが Before との違いです。Before はいったん `const tasks` で受け取り、そのあと加工していました。ただし N+1 を消しているのは `return` の位置ではありません。`const` で受けてから返す形でも、その間に問い合わせを足せば N+1 は起きます。効いているのは次のコードブロックの `select` に `project` を入れ子で書いて関連するプロジェクト名も取得する点です。内部で発行される SQL が 1 本になるとは限りません。`where` の中身は Before と変えていません。

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

select: {
  id: true,
  completedAt: true,
  status: true,
  priority: true,
  project: {
    select: {
      id: true,
      name: true,
    },
  },
},
});
}
```

このコードの強み:

- タスクとプロジェクト情報を Prisma にまとめて取得させるので問い合わせ回数が読みやすい
- `projectId` を手で持ち回らず、戻り値の形が「画面で使うデータ」に近くなります
- 担当者や関連情報を足すときも `select` / `include` の中に集約でき、取得ロジックが散らばりにくい

覚えておきたいエッセンス

一覧やレポートで `relation` を使うなら 1 件ずつ取得する前に Prisma の `select` / `include` でまとめて取れないかを考えます。

完成コード全体

今日は3つのファイルを触りました。サーバーへ手続きを1本足し、新しいページを1枚作り、すでにあるページへリンクを1つ差し込む、という3方向の作業が混ざっています。どこへ何を貼ったか分からなくなった場合は以下のコードで各ファイルを丸ごと置き換えてください。ファイルごとに、上から順に読めば Step 0 から Step 6 で書いた断片がどう1つにまとまったかを確認られます。

ファイル	役割	対応する Step
<code>src/server/api/routers/report.ts</code>	レポート集計 API。 <code>getOverview</code> の下に <code>getWeeklyReport</code> を 足す	Step 0
<code>src/app/report/weekly/page.tsx</code>	週次レポート画面。サマ リーカード3枚とグラ フ3枚	Step 4~ Step 6
<code>src/app/report/page.tsx</code>	レポート画面。週次レ ポートへのリンクを 足す	Step 4

```
src/server/api/routers/report.ts
```

import 群:

```
// filepath: src/server/api/routers/report.ts
// 完成版: import 群
import { TRPCError } from '@trpc/server';
import { z } from 'zod';
import { TASK_PRIORITY } from '@lib/constant/priority';
import { USER_ROLE } from '@lib/constant/roles';
import { TASK_STATUS } from '@lib/constant/status';
import { prisma } from '@lib/prisma';
import { createTRPCRouter, protectedProcedure } from
'../trpc';
```

Day 21 の時点では4つの import だけでした。今日足したのは `TRPCError` ・ `z` ・ `TASK_PRIORITY` ・ `USER_ROLE` の4つで、どれも `getWeeklyReport` だけが使います。使う予定の無い import を先に書かないのは Biome が未使用の名前をエラーとして報告するからです。実際に使う日まで待って足せば警告を抱えたまま次の日へ進まずに済みます。

getOverview の入口:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: getOverview の入口
export const reportRouter = createTRPCRouter({
  getOverview: protectedProcedure.query(async ({ ctx
})) => {
    const userId = ctx.session.userId;
```

Day 23: 週次レポートを表示しよう

入口でログイン中の `userId` を受け取ります。この値を12本の問い合わせへ含め、取得時点で参加しているプロジェクトだけを集計します。参加中のプロジェクトが0件なら、各問い合わせが0または空配列を返し、最後の `return` が同じ形の空レポートを組み立てます。

集計範囲を決める2つの条件:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 集計範囲を決める2つの条件

// アーカイブ済みプロジェクトのタスクは集計対象外にし、
// プロジェクト数・統計との整合を取る。

const projectScope = {
  project: {
    isArchived: false,
    members: { some: { userId } },
  },
} as const;

// ダッシュボードの「アクティブな作業」を母数とするため、
// CANCELLED は集計から除外する。

const activeTasksFilter = {
  ...projectScope,
  NOT: { status: TASK_STATUS.CANCELLED },
} as const;
```


条件を変数にしておくとそのあとの12本の問い合わせで同じ絞り込みを使い回せます。1本ずつ条件を書き写す形にするとあとで除外の決まりを変えたときに直し漏れが出てカードの合計と表の内訳が合わなくなります。`as const` を付けているのはこの2つを書き換えないと決めた印です。

Promise.all の受け取り側:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 12個の集計結果を受け取る

const [
  projects,
  totalTasks,
  completedTasks,
  inProgressTasks,
  inReviewTasks,
  todoTasks,
  totalTimeAggregate,
  recentTasks,
  statusGroups,
  priorityGroups,
  projectTaskGroups,
  projectDoneGroups,
] = await Promise.all([
```

左辺の12個の名前はこのあと並べる問い合わせと同じ順番で受け取ります。件数どうしを入れ替えても型は合ってしまうので間違えてもエラーは出ません。画面には形だけ正しい別の数字が並びます。集計を足すときは左辺と問い合わせの両方の末尾へ足してください。

Day 23: 週次レポートを表示しよう

問い合わせの前半:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: プロジェクト一覧と件数の集計

prisma.project.findMany({
  where: {
    isArchived: false,
    members: { some: { userId } },
  },
  select: { id: true, name: true },
  orderBy: { createdAt: 'desc' },
}),
prisma.task.count({ where: activeTasksFilter }),
prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.DONE,
  },
}),
prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.IN_PROGRESS,
  },
}),
```

総タスク数だけ `activeTasksFilter` を使い、ステータス別の件数は `projectScope` を使います。母数からは中止したタスクを外し、内訳では実際のステータスをそのまま数える分担です。`select` で `id` と `name` だけを取るのは画面で使う項目がこの2つに限られるからです。

問い合わせの中盤:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 残りの件数と作業時間の合計

prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.IN_REVIEW,
  },
}),

prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.TODO,
  },
}),

prisma.task.aggregate({
  where: activeTasksFilter,
  _sum: { timeSpentMinutes: true },
}),
```

Day 23: 週次レポートを表示しよう

aggregate は合計や平均をデータベース側で出す関数です。全タスクを取り出してから足し算する書き方でも答えは同じですがその場合は件数ぶんのデータがサーバーのメモリへ載ります。合計だけが欲しいので数える仕事はデータベースに任せます。

直近タスクとステータス別集計:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 直近5件とステータス・優先度の集計

prisma.task.findMany({
  where: activeTasksFilter,
  select: {
    id: true,
    title: true,
    status: true,
    priority: true,
  },
  orderBy: { updatedAt: 'desc' },
  take: 5,
}),

prisma.task.groupBy({
  by: ['status'],
  where: activeTasksFilter,
  _count: { _all: true },
}),

prisma.task.groupBy({
  by: ['priority'],
  where: activeTasksFilter,
  _count: { _all: true },
}),
```

`groupBy` は「同じ値ごとにまとめて数える」関数で、ステータス別と優先度別の件数をここで作ります。Day 22 の円グラフが受け取っているのはこの2本の結果です。

`take: 5` を付けた `findMany` だけが実際の行を返しますがそれも直近5件に限っているので返るデータ量は件数が増えても変わりません。

Day 23: 週次レポートを表示しよう

プロジェクト別集計と問い合わせの締め:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: プロジェクト別の集計2本

prisma.task.groupBy({
  by: ['projectId'],
  where: activeTasksFilter,
  _count: { _all: true },
  _sum: { timeSpentMinutes: true },
}),
prisma.task.groupBy({
  by: ['projectId'],
  where: {
    ...projectScope,
    status: TASK_STATUS.DONE,
  },
  _count: { _all: true },
}),
]);
```

プロジェクト別の集計を2本に分けているのは絞り込みの条件が違うからです。1本目は母数と作業時間、2本目は完了数だけを数えます。1本にまとめて完了かどうかも軸に加えると完了が0件のプロジェクトの行が結果から消え、あとで組み立てるときに扱いが増えます。

表示用の値と Map の組み立て:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 合計値の計算と辞書づくり

const totalTimeSpent =
totalTimeAggregate._sum.timeSpentMinutes ?? 0;

const averageTimePerTask = totalTasks > 0 ?
totalTimeSpent / totalTasks : 0;

const completionRate = totalTasks > 0 ? Math.round((
completedTasks / totalTasks) * 100) : 0;

const doneCountByProject = new Map(
  projectDoneGroups.map((group) => [group.projectId,
group._count._all]),
);

const taskStatsByProject = new Map(
  projectTaskGroups.map((group) => [
    group.projectId,
    {
      totalTasks: group._count._all,
      totalTimeSpent: group._sum.timeSpentMinutes ??
0,
    },
  ]),
);
```

Day 23: 週次レポートを表示しよう

平均と完了率で `totalTasks > 0` を先に見ているのは0 で割った答えが `NaN` になるからです。画面には数字ではなく「NaN%」という文字が出ます。`Map` に変えているのは次のブロックでプロジェクト1件ずつに集計を引き当てるためです。配列のまま毎回 `find` を回すとプロジェクトが増えるほど探す回数が掛け算で増えます。

`getOverview` の戻り値の前半:


```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 画面へ返す値の前半

return {
  totalProjects: projects.length,
  totalTasks,
  completedTasks,
  inProgressTasks,
  inReviewTasks,
  todoTasks,
  completionRate,
  totalTimeSpent,
  averageTimePerTask,
  recentTasks,
  statusData: statusGroups.map((group) => ({
    key: group.status,
    value: group._count._all,
  })),
  priorityData: priorityGroups.map((group) => ({
    key: group.priority,
    value: group._count._all,
  })),
}
```

`groupBy` の結果は `_count._all` という深い形をしています。それをそのまま返すと画面側が Prisma の都合に合わせて書かれてしまいます。ここで `{ key, value }` へ並べ替えておくと Day 22 の円グラフは受け取った配列に `name` を足すだけで済みます。

projectStats の組み立て:

Day 23: 週次レポートを表示しよう

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: プロジェクト別統計と getOverview の締め

    projectStats: projects.map((project) => {
        const taskStats = taskStatsByProject.get(
project.id) ?? {
            totalTasks: 0,
            totalTimeSpent: 0,
        };
        const completedTaskCount = doneCountByProject.get(
project.id) ?? 0;

        return {
            id: project.id,
            name: project.name,
            totalTasks: taskStats.totalTasks,
            completedTasks: completedTaskCount,
            progress:
                taskStats.totalTasks > 0 ? (
completedTaskCount / taskStats.totalTasks) * 100 : 0,
            totalTimeHours: taskStats.totalTimeSpent / 60,
        };
    }),
};
},
```

起点をプロジェクト一覧にしているのでタスクが1件も無いプロジェクトの行が残ります。集計結果の側から作るとその行は結果に現れず、作ったばかりのプロジェクトが表から消えます。`?? { totalTasks: 0, totalTimeSpent: 0 }`はそのタスクが無いプロジェクトのための受け皿です。

getWeeklyReport の入力と認可:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: getWeeklyReport の入力検証と権限確認

getWeeklyReport: protectedProcedure
  .input(
    z.object({
      weeks: z.number().int().min(1).max(12).default(4),
      userId: z.string().cuid().optional(),
    }).default({}),
  )
  .query(async ({ ctx, input }) => {
    if (input.userId && input.userId !==
ctx.session.userId) {
      if (ctx.session.role !== USER_ROLE.ADMIN) {
        throw new TRPCError({
          code: 'FORBIDDEN',
          message: '管理者権限が必要です',
        });
      }
    }
  })
```

Day 23: 週次レポートを表示しよう

入力の範囲をここで決めてしまうと画面側は値の確かめ方を持たずに済み
ます。`.min(1).max(12)` を通らない値は tRPC が先に弾き、データベースまで届
きません。権限の確認も入口に置きます。取得したあとで確かめる形にすると返さな
いと決めたデータを一度は読み出すことになります。

集計期間の決め方:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)

// 完成版: 週バケットの上端と下端

    const targetUserId = input.userId ??
ctx.session.userId;

    const now = new Date();

// 週バケットは「今日で終わる直近7日間」を最終週として7日刻みで遡る。

// 旧実装は最終週が「今日0時～現在」だけの進行中バケットで、「4週間」表示の

// 実カバ範囲が3週間+今日に縮んでいた。排他的上端を

// 明日0時に固定した完全な7日バケット×weeks本にすることで、ラベル・週平均の

// 分母と実際の集計範囲が一致し、範囲内タスクは必ずいずれかの週に入る。

// 日付ラベルは toISOString() (UTC)で出すため、バケット境界も UTC で刻む。

// ローカル時刻メソッドで刻むと、JST などのサーバーでラベルが1日ずれる。

    const rangeEnd = new Date(now);

    rangeEnd.setUTCHours(0, 0, 0, 0);

    rangeEnd.setUTCDate(rangeEnd.getUTCDate() + 1);

    const startDate = new Date(rangeEnd);

    startDate.setUTCDate(startDate.getUTCDate() -
input.weeks * 7);
```

Day 23: 週次レポートを表示しよう

上端を明日の 00:00 UTC にそろえ、そこから週数ぶん遡ります。これで各週を UTC の日付境界で区切れます。取得するのは現在までのタスクなので最後の週には今日の途中までが入ります。上端を今にすると下端も同じ時刻までずれ、4 週間なら直近 28 日間の集計になります。

絞り込み条件と1回の取得:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 対象タスクをまとめて取る

const where = {
  status: TASK_STATUS.DONE,
  completedAt: { gte: startDate, lte: now },
  assigneeId: targetUserId,
  project: { members: { some: { userId:
targetUserId } } },
};

const tasks = await prisma.task.findMany({
  where,
  select: {
    id: true,
    completedAt: true,
    status: true,
    priority: true,
  },
});
```

期間全体を1回で取り、週への仕分けはこのあとの `filter` に任せます。週ごとに問い合わせる書き方だと12週間を選んだときに12往復します。読み込む行数は変わらないのに往復の回数だけが週数に比例して増えます。`where` を1か所にまとめているので週ごとの絞り込みは日付の比較だけで済みます。

週ごとのバケットとステータス集計:

Day 23: 週次レポートを表示しよう

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 週ごとの箱づくりとステータス別件数

const weeklyData = Array.from({ length: input.weeks
}, (_, i) => {

  const weekStart = new Date(startDate);
  weekStart.setUTCDate(weekStart.getUTCDate() + i *
7);

  const weekEnd = new Date(weekStart);
  weekEnd.setUTCDate(weekEnd.getUTCDate() + 7);

  const weekTasks = tasks.filter(
    (task) => task.completedAt && task.completedAt
>= weekStart && task.completedAt < weekEnd,
  );

  return {
    week: `${i + 1}週目`,
    weekStart: weekStart.toISOString().split('T')[
0],
    totalCompleted: weekTasks.length,
    byStatus: Object.fromEntries(
      Object.values(TASK_STATUS).map((status) => [
        status,
        weekTasks.filter((t) => t.status ===
status).length,
      ]),
    ),
  },
}
```


終わりを `< weekEnd` にしてあるので境界ちょうどに完了したタスクが2つの週で二重に数えられません。`Array.from` で先に週数ぶんの箱を作るのは完了が0件の週も配列へ残すためです。実際にあった週だけを作る書き方にとするとグラフの横軸から静かな週が抜け落ち、推移が実際より詰まって見えます。

優先度集計と `getWeeklyReport` の戻り値:

Day 23: 週次レポートを表示しよう

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 優先度別件数と返す値

    byPriority: Object.fromEntries(
        Object.values(TASK_PRIORITY).map((
priority) => [
            priority,
            weekTasks.filter((t) => t.priority ===
priority).length,
            ],
        ),
    );
});

return {
    weeks: input.weeks,
    startDate: startDate.toISOString(),
    endDate: now.toISOString(),
    weeklyData,
    totalCompleted: tasks.length,
};
},
});
```

`Object.values(TASK_STATUS)` と `Object.values(TASK_PRIORITY)` を回しているのなのであとで種類を1つ増やしてもこの集計は定数を直すだけで追従します。ここで `'DONE'` などを直に並べると増えた種類がグラフから静かに抜けます。最後の `});` で `reportRouter` 全体が閉じ、手続きは `getOverview` と `getWeeklyReport` の2本立てになります。

`src/app/report/weekly/page.tsx`

日付とページ枠の取り込み:

```
// filepath: src/app/report/weekly/page.tsx
// 完成版: 日付とページ枠の取り込み
'use client';
import { formatDateOnly } from '@lib/date';
import { useState } from 'react';
import { AppLayout }
  from '@component/layout/app-layout';
import {
  Card, CardContent,
  CardHeader, CardTitle,
} from '@component/ui/card';
import { PageLoadingSpinner }
  from '@component/ui/loading-spinner';
```

先頭の `'use client'` が無いと `useState` を書いた行でフックはサーバー側で使えないというエラーになります。`formatDateOnly` は配布済みの `src/lib/date.ts` にある関数です。UTC の日時文字列から日付部分を取り出すのでブラウザのタイムゾーンが変わっても集計期間の日付を保てます。

選択欄とグラフ部品の取り込み:

Day 23: 週次レポートを表示しよう

```
// filepath: src/app/report/weekly/page.tsx(同じファイルの続き)
// 完成版: 選択欄とグラフ部品の取り込み

import {
  Select, SelectContent,
  SelectItem, SelectTrigger,
  SelectValue,
} from '@component/ui/select';
import {
  Bar, BarChart, CartesianGrid,
  Legend, Line, LineChart,
  ResponsiveContainer,
  Tooltip, XAxis, YAxis,
} from 'recharts';
import {
  TASK_PRIORITY, TASK_PRIORITY_COLORS,
} from '@lib/constant/priority';
import { api } from '@trpc/react';

const CHART_PRIMARY_COLOR = '#8884d8';
```

Recharts から取り込む10個のうち、`CartesianGrid`・`XAxis`・`YAxis`・`LineChart`・`Line`・`BarChart`・`Bar` は今日が初登場です。Day 22 は円グラフだけだったので、軸・折れ線・棒の部品は要りませんでした。優先度の定数を色つきで取り込むのはDay 22 の円グラフと同じ色で棒を塗るためです。`CHART_PRIMARY_COLOR` だけは対応表を持たない折れ線1本ぶんの色なのでこの画面の定数として1か所に置きます。

関数の入口とデータ取得:

```
// filepath: src/app/report/weekly/page.tsx(同じファイルの続き)
// 完成版: 関数の入口とデータ取得

export default function WeeklyReportPage() {
  const [weeks, setWeeks] = useState('4');

  const {
    data: reportData,
    isLoading,
  } = api.report.getWeeklyReport.useQuery({
    weeks: Number.parseInt(weeks, 10),
  });

  if (isLoading) {
    return <PageLoadingSpinner />;
  }
}
```

`weeks` を文字列で持つのは `Select` が文字列でしか値をやり取りしないからです。サーバー側の `weeks` は数値なので `useQuery` へ渡すところで `Number.parseInt` を挟みます。`weeks` が変わると `useQuery` は新しい引数として扱い、自動でもう一度取りに行きます。取得の指示をどこにも書かずに済むのはこの仕組みのおかげです。

グラフへ渡す2つの配列:

Day 23: 週次レポートを表示しよう

```
// filepath: src/app/report/weekly/page.tsx(同じファイルの続き)
// 完成版: グラフへ渡す2つの配列

const chartData =
  reportData?.weeklyData.map((week) => ({
    name: week.week,
    completed: week.totalCompleted,
    high:
      week.byPriority[TASK_PRIORITY.HIGH] ?? 0,
    urgent:
      week.byPriority[TASK_PRIORITY.URGENT]
        ?? 0,
  }));

const priorityData = reportData?.weeklyData.map((
week) => ({
  name: week.week,
  low: week.byPriority[TASK_PRIORITY.LOW] ?? 0,
  medium: week.byPriority[TASK_PRIORITY.MEDIUM] ?? 0,
  high: week.byPriority[TASK_PRIORITY.HIGH] ?? 0,
  urgent: week.byPriority[TASK_PRIORITY.URGENT] ?? 0,
}));
```

Recharts は「1週分が1オブジェクト、系列名がそのキー」という形の配列を求めます。サーバーが返す `byPriority` は入れ物が1段深いのでここで平らな形へ組み替えます。2つの配列で `name` というキーをそろえてあるのは3枚のグラフが同じ `XAxis dataKey="name"` を使えるようにするためです。

見出しの行:

```
// filepath: src/app/report/weekly/page.tsx(同じファイルの続き)
// 完成版: 見出しの行

return (
  <AppLayout>
    <div className="space-y-6">
      <div className="flex items-center
        justify-between">
        <h1 className="text-3xl font-bold">
          週次レポート
        </h1>
```

見出しの行を `flex` にして `justify-between` を付けると中の要素が左端と右端に分かれます。だから見出しの次に置く週数の選択欄が自動で右端へ寄ります。この指定が無いと選択欄は見出しの真下へ回り込み、縦に2行ぶんの場所を取ります。

週数の選択欄:

Day 23: 週次レポートを表示しよう

```
    { /* filepath: src/app/report/weekly/page.tsx(同じファイルの続き) */ }

    { /* 完成版: 週数の選択欄 */ }

    <div className="w-[150px]">
      <Select
        value={weeks}
        onChange={setWeeks}>
        <SelectTrigger>
          <SelectValue placeholder="期間" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="4">
            4週間
          </SelectItem>
          <SelectItem value="8">
            8週間
          </SelectItem>
          <SelectItem value="12">
            12週間
          </SelectItem>
        </SelectContent>
      </Select>
    </div>
  </div>
```


`onValueChange` に `setWeeks` をそのまま渡しているので選んだ値が状態へ入り、`useQuery` の引数が変わって取り直しが始まります。選択肢を 4・8・12 の 3 つに絞ってあるのはサーバー側の `.min(1).max(12)` の範囲に収めるためです。自由に数字を入れる欄にすると 13 を入れた読者が入力エラーだけを受け取ります。

完了タスク合計のカード:

```

    {/* filepath: src/app/report/weekly/page.tsx(同じ
ファイルの続き) */}

    {/* 完成版: 完了タスク合計のカード */}

    <div className="grid grid-cols-1
      md:grid-cols-3 gap-4">
      <Card>
        <CardContent className="pt-6">
          <p className="text-sm
            text-muted-foreground mb-1">
            完了タスク合計
          </p>
          <p className="text-3xl font-bold">
            {reportData?.totalCompleted ?? 0}
          </p>
        </CardContent>
      </Card>

```

`?? 0` は数字の場所が空欄になるのを防ぐだけの守りです。取得が失敗したときもここは 0 と表示されるので「完了 0 件」との区別は付きません。区別を付けたい場合は `useQuery` から `error` も受け取り、失敗したときだけ別の文言を出す形にします。

Day 23: 週次レポートを表示しよう

週平均のカード:

```
    { /* filepath: src/app/report/weekly/page.tsx(同じファイルの続き) */ }
    { /* 完成版: 週平均のカード */ }

    <Card>
      <CardContent className="pt-6">
        <p className="text-sm text-muted-foreground mb-1">
          週平均
        </p>
        <p className="text-3xl font-bold">
          {reportData?.totalCompleted
            ? Math.round(
                reportData.totalCompleted
                  / Number.parseInt(weeks, 10)
              )
            : 0}
        </p>
      </CardContent>
    </Card>
```

API に渡す週数と平均の分母は同じ `weeks` から作ります。`Math.round` は計算結果を整数に丸めるための関数です。今日の途中までの件数を含むため週平均もこの時点での値です。

対象期間のカード:

```

    {/* filepath: src/app/report/weekly/page.tsx(同
じファイルの続き) */}

    {/* 完成版: 対象期間のカード */}

    <Card>
      <CardContent className="pt-6">
        <p className="text-sm
          text-muted-foreground mb-1">
          集計期間(UTC基準)</p>
        <p className="text-lg font-semibold">
          {reportData?.startDate
            && reportData?.endDate
              ? `${formatDateOnly(
reportData.startDate)} - ${formatDateOnly(
reportData.endDate)}`
                : '-'}
          </p>
        </CardContent>
      </Card>
    </div>

```

2つの日付がそろっているときだけ期間を組み立て片方でも無ければ `'-'` を出します。開始日と終了日がそろうまで期間の表示を待つためです。このカードだけ `text-lg` にしてあるのは日付2つを並べた文字列が `text-3xl` では折り返すからです。

週別完了タスク数の折れ線グラフ:

Day 23: 週次レポートを表示しよう

```
    { /* filepath: src/app/report/weekly/page.tsx(同じ
ファイルの続き) */ }

    { /* 完成版: 週別完了タスク数の折れ線グラフ */ }

    <div className="grid grid-cols-1
      lg:grid-cols-2 gap-6">
      <Card className="col-span-1 lg:col-span-2">
        <CardHeader>
          <CardTitle>週別完了タスク数</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="h-[300px]">
            <ResponsiveContainer width="100%"
height="100%">
              <LineChart data={chartData ?? []}>
                <CartesianGrid strokeDasharray="3 3"
/>

                <XAxis dataKey="name" />
                <YAxis /><Tooltip /><Legend />
                <Line type="monotone"
                  dataKey="completed"
                  stroke={CHART_PRIMARY_COLOR}
                  name="完了数" strokeWidth={2} />
              </LineChart>
            </ResponsiveContainer>
          </div>
        </CardContent>
      </Card>
```

このカードだけ `lg:col-span-2` で2列ぶんの幅を取っています。折れ線で見せたいのは件数そのものではなく増減の向きなので横に長いほうが向きを読み取りやすくなります。`chartData ?? []` の `?? []` は `chartData` が `undefined` のときに `LineChart` へ何も渡らない状態を避けるための既定値です。

高・緊急の完了タスク数の棒グラフ:

Day 23: 週次レポートを表示しよう

```
    { /* filepath: src/app/report/weekly/page.tsx(同じファイルの続き) */ }

    { /* 完成版: 高・緊急の完了タスク数の棒グラフ */ }

    <Card>
      <CardHeader>
        <CardTitle>
          高・緊急の完了タスク数</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="h-[300px]">
            <ResponsiveContainer width="100%"
height="100%">
              <BarChart data={chartData ?? []}>
                <CartesianGrid strokeDasharray="3 3"
/>

                <XAxis dataKey="name" />
                <YAxis /><Tooltip /><Legend />
                <Bar dataKey="urgent" name="緊急"
                  fill={TASK_PRIORITY_COLORS.URGENT}
/>

                <Bar dataKey="high" name="高"
                  fill={TASK_PRIORITY_COLORS.HIGH} />
              </BarChart>
            </ResponsiveContainer>
          </div>
        </CardContent>
      </Card>
```

`Bar` が2本だけなのは `chartData` に `urgent` と `high` しか入れていないからです。週次レポートで確かめたいのは急ぎの仕事の片づき方なので低と中の件数は落としてあります。4段階すべての内訳は次の積み上げ棒グラフで見られます。

`fill` を定数から引くのは色を1か所で管理するためです。

完了タスクの優先度別内訳の枠:

```

    { /* filepath: src/app/report/weekly/page.tsx(同
    じファイルの続き) */}

    { /* 完成版: 完了タスクの優先度別内訳の枠 */}

    <Card>
      <CardHeader>
        <CardTitle>
          完了タスクの優先度別内訳</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="h-[300px]">
            <ResponsiveContainer width="100%"
            height="100%">
              <BarChart data={priorityData ?? []}>
                <CartesianGrid strokeDasharray="3 3"
                />
                <XAxis dataKey="name" />
                <YAxis /><Tooltip /><Legend />
              </BarChart>
            </ResponsiveContainer>
          </div>
        </CardContent>
      </Card>
    
```

Day 23: 週次レポートを表示しよう

外側の作りは優先度グラフとそろえ、`data` だけを `priorityData` に変えます。2つの配列はどちらも週ラベルを `name` へ入れてあります。だから `XAxis` の指定は書き換えずに済みます。`h-[300px]` は3枚とも同じ値です。3枚を横へ並べても高さがそろいます。

完了タスクの優先度別内訳の積み上げと閉じタグ:


```

        { /* filepath:
src/app/report/weekly/page.tsx(同じファイルの続き) */ }
        { /* 完成版: 積み上げの4本と閉じタグ
*/ }

        <Bar dataKey="low" stackId="priority"
name="低"

        fill={TASK_PRIORITY_COLORS.LOW} />
        <Bar dataKey="medium" stackId="priority"
name="中"

        fill={TASK_PRIORITY_COLORS.MEDIUM} />
        <Bar dataKey="high" stackId="priority"
name="高"

        fill={TASK_PRIORITY_COLORS.HIGH} />
        <Bar dataKey="urgent" stackId="priority"
name="緊急"

        fill={TASK_PRIORITY_COLORS.URGENT} />
        </BarChart>
        </ResponsiveContainer>
        </div>
        </CardContent>
        </Card>
    </div>
</div>
</AppLayout>
);
}

```

Day 23: 週次レポートを表示しよう

4本の `Bar` に同じ `stackId` を付けると横に並ばず1本の棒として積み上がります。週ごとの合計と内訳を同時に読ませたいので並べるのではなく積みます。

`stackId` を1つでも書き忘れるとその系列だけが隣に独立した棒として立ちます。最後の閉じタグは `BarChart` から `AppLayout` まで開いた順の逆にとどります。

`src/app/report/page.tsx`

このファイルで今日書き換えたのは先頭の `import` 2行と見出しの行だけです。ただし見出しの行はタグの入れ子が1段深くなるため貼る場所を間違えると統計カードまで巻き込みます。以下は Day 22 の終わりの状態にリンクを足した全文で、ブロックの区切りは Day 22 の「完成コード全体」とそろえてあります。2つを並べるとどこが増えたかを行単位で見比べられます。

画面部品の取り込み:

```
// filepath: src/app/report/page.tsx
// 完成版: 画面部品の取り込み
'use client';

import { ArrowRight } from 'lucide-react';
import Link from 'next/link';
import {
  Cell, Legend, Pie, PieChart,
  ResponsiveContainer, Tooltip,
} from 'recharts';
import { AppLayout } from '@component/layout/app-layout';
import {
  Card, CardContent,
  CardHeader, CardTitle,
} from '@component/ui/card';
import { PageLoadingSpinner } from
  '@component/ui/loading-spinner';
import {
  Table, TableBody, TableCell,
  TableHead, TableHeader, TableRow,
} from '@component/ui/table';
```

今日足したのは `ArrowRight` と `Link` の2行です。`recharts` より前に並ぶのは Biome が外部ライブラリをアルファベット順に置く決まりだからです。手元で末尾に足していても `npm run fix` でここへ移るので差分を見て驚かないでください。

色とラベルの定数:

Day 23: 週次レポートを表示しよう

```
// filepath: src/app/report/page.tsx(同じファイルの続き)
// 完成版: 色とラベルの定数、通信の入口
import {
  isTaskPriority,
  TASK_PRIORITY_COLORS,
  TASK_PRIORITY_LABELS,
} from '@/lib/constant/priority';
import {
  isTaskStatus,
  TASK_STATUS_COLORS,
  TASK_STATUS_LABELS,
} from '@/lib/constant/status';
import { api } from '@trpc/react';

const CHART_FALLBACK_COLOR = '#9e9e9e';
```

ここは Day 22 のままです。週次ページも同じ定数から色を引くので2つの画面で同じステータスが同じ色になります。片方の画面だけ色を直に書くと行き来したときに同じ「完了」が違う色で出て読者は別のものを見ていると受け取ります。

取得と表示用の値づくり:

```
// filepath: src/app/report/page.tsx
// 完成版: 取得と表示用の値づくり
export default function ReportPage() {
  const { data: overview, isLoading } =
    api.report.getOverview.useQuery();

  const totalTasks = overview?.totalTasks ?? 0;
  const completionRate =
    overview?.completionRate ?? 0;
  const totalTimeHours =
    ((overview?.totalTimeSpent ?? 0) / 60)
      .toFixed(1);
  const averageTimeHours =
    ((overview?.averageTimePerTask ?? 0) / 60)
      .toFixed(1);
```

呼んでいるのは `getOverview` だけで、今日足した `getWeeklyReport` はこのページから呼びません。1つの画面に両方の集計を載せると週次のためだけに待ち時間が伸びます。見たい人だけがリンクをたどる形にしてレポート画面の表示は Day 22 と同じ速さのまま保ちます。

円グラフへ渡す2つの配列:

Day 23: 週次レポートを表示しよう

```
// filepath: src/app/report/page.tsx(同じファイルの続き)
// 完成版: 円グラフへ渡す2つの配列

const statusData =
  overview?.statusData.map((entry) => ({
    ...entry,
    name: isTaskStatus(entry.key)
      ? TASK_STATUS_LABELS[entry.key]
      : entry.key,
  }))) ?? [];

const priorityData =
  overview?.priorityData.map((entry) => ({
    ...entry,
    name: isTaskPriority(entry.key)
      ? TASK_PRIORITY_LABELS[entry.key]
      : entry.key,
  }))) ?? [];

if (isLoading) {
  return <PageLoadingSpinner />;
}
```

この2つも Day 22 のままです。週次ページの `chartData` とは形が違います。こちらは1件が1つのステータス、あちらは1件が1週を表します。2つの画面では配列の1件が表す対象も違います。その違いを頭に入れておくとグラフが空のときにどちらの組み替えを見ればよいか迷いません。

見出しとリンクの行:

```
// filepath: src/app/report/page.tsx
// 完成版: 見出しと週次レポートへのリンク

return (
  <AppLayout>
    <div className="space-y-6">
      <div className="flex flex-col gap-4
        items-start sm:flex-row
        sm:items-center sm:justify-between">
        <div>
          <h1 className="text-3xl font-bold
            tracking-tight">
            レポート・統計
          </h1>
          <p className="text-muted-foreground">
            プロジェクトの進捗とタスクの状況を確認できます。
          </p>
        </div>
        <Link
          href="/report/weekly"
          className="inline-flex items-center gap-2
            text-sm font-medium text-primary
            hover:underline"
          >
          週次レポートを見る
          <ArrowRight className="h-4 w-4" />
        </Link>
      </div>
    </div>
  </AppLayout>
)
```

Day 23: 週次レポートを表示しよう

今日の変更はここだけです。Day 22 では見出しと説明文を包む `div` が1つでしたがリンクを右へ置くために外側の `div` を1つ増やし、見出しの組と `Link` を横に並べています。狭い画面では `flex-col` で縦積みになり、`sm` 以上で横並びに切り替わります。リンクを見出しの組の中へ入れてしまうと説明文の下に潜って気づかれませんか。

統計カードの前半:


```

    { /* filepath: src/app/report/page.tsx(同じファイルの続き) */ }
    { /* 完成版: 統計カード(タスク数・完了率) */ }
    <div className="grid grid-cols-1
      sm:grid-cols-2 lg:grid-cols-4 gap-4">
      <Card>
        <CardContent className="pt-6">
          <p className="text-sm
            text-muted-foreground mb-1">
            タスク数</p>
          <p className="text-3xl font-bold">
            {totalTasks}</p>
        </CardContent>
      </Card>
      <Card>
        <CardContent className="pt-6">
          <p className="text-sm
            text-muted-foreground mb-1">
            完了率</p>
          <p className="text-3xl font-bold">
            {completionRate}%</p>
        </CardContent>
      </Card>
    </div>

```

このグリッドは前のブロックで閉じた見出しの `div` の外側に並びます。リンクを足すときに閉じタグを1つ落とすとカードのグリッドが見出しの `div` の中へ入り、右端のリンクの隣に4枚のカードが押し込まれます。カードが急に細くなったときは見出しの `</div>` の位置を確かめてください。

Day 23: 週次レポートを表示しよう

統計カードの後半:

```
    { /* filepath: src/app/report/page.tsx(同じファイルの続き) */ }
    { /* 完成版: 統計カード(作業時間の合計と平均) */ }

    <Card>
      <CardContent className="pt-6">
        <p className="text-sm text-muted-foreground mb-1">
          合計作業時間</p>
        <p className="text-3xl font-bold">
          {totalTimeHours}h</p>
      </CardContent>
    </Card>
    <Card>
      <CardContent className="pt-6">
        <p className="text-sm text-muted-foreground mb-1">
          平均作業時間/タスク</p>
        <p className="text-3xl font-bold">
          {averageTimeHours}h</p>
      </CardContent>
    </Card>
  </div>
```

作業時間の2枚が持っている値は期間で切らない全部の合計です。週次ページの「完了タスク合計」とは数え方が違います。2つの画面で数字が食い違うのは正しい状態です。同じ数を並べたい場合は2つの画面を同じ期間で切る必要があります。

ステータス円グラフの枠:

```
    { /* filepath: src/app/report/page.tsx(同じファイ  
ルの続き) */ }  
    { /* 完成版: ステータス円グラフの枠 */ }  
    <div className="grid grid-cols-1  
      md:grid-cols-2 gap-6">  
      <Card>  
        <CardHeader>  
          <CardTitle>ステータス別タスク</CardTitle>  
        </CardHeader>  
        <CardContent>  
          <div className="h-[300px]">  
            <ResponsiveContainer  
              width="100%" height="100%">  
              <PieChart>  
                <Pie  
                  data={statusData}  
                  dataKey="value"  
                  nameKey="name"  
                  cx="50%"  
                  cy="50%"  
                  outerRadius={80}  
                  label  
                >
```

`h-[300px]` の `div` が高さの基準になる形は週次ページの3枚のグラフと同じです。高さの数字までそろえてあるので2つの画面を行き来してもグラフの大きさが変わりません。この基準の `div` を外すと `height="100%"` が 0 と計算され、扇は1枚も描かれません。

ステータス円グラフの色付け:

```

        {/* filepath:
src/app/report/page.tsx(同じファイルの続き) */}
        {/* 完成版: ステータス円グラフの色と
閉じタグ */}

        {statusData.map((entry) => (
          <Cell
            key={entry.key}
            fill={
              isTaskStatus(entry.key)
                ? TASK_STATUS_COLORS[
entry.key]
                : CHART_FALLBACK_COLOR
            }
          />
        ))}
      </Pie>
      <Tooltip />
      <Legend />
    </PieChart>
  </ResponsiveContainer>
</div>
</CardContent>
</Card>

```

Day 23: 週次レポートを表示しよう

型ガードを挟んでから対応表を引く形は週次ページの

`fill={TASK_PRIORITY_COLORS.HIGH}` とは書き方が違います。あちらは書く時点で優先度が決まっているので確認が要りません。こちらはサーバーから届いた文字列で引くためその文字列が対応表のキーだと確かめる手順が入ります。

優先度円グラフの枠:

```
    { /* filepath: src/app/report/page.tsx(同じファイルの続き) */ }
    { /* 完成版: 優先度円グラフの枠 */ }
    <Card>
      <CardHeader>
        <CardTitle>優先度別タスク</CardTitle>
      </CardHeader>
      <CardContent>
        <div className="h-[300px]">
          <ResponsiveContainer
            width="100%" height="100%">
            <PieChart>
              <Pie
                data={priorityData}
                dataKey="value"
                nameKey="name"
                cx="50%"
                cy="50%"
                outerRadius={80}
                label
              >
```

この円グラフは優先度 4 段階すべてを扇にします。週次ページの棒グラフが緊急と高の 2 本だけなのはあちらが急ぎの仕事の推移を見る画面だからです。全体の割合はこちら、週ごとの推移はあちらと役割を分けてあるのでどちらかへ寄せる必要はありません。

優先度円グラフの色付け:

Day 23: 週次レポートを表示しよう

```
        { /* filepath:
src/app/report/page.tsx(同じファイルの続き) */ }

        { /* 完成版: 優先度円グラフの色と閉じ
タグ */ }

        {priorityData.map((entry) => (
            <Cell
                key={entry.key}
                fill={
                    isTaskPriority(entry.key)
                        ? TASK_PRIORITY_COLORS[
entry.key]
                        : CHART_FALLBACK_COLOR
                }
            />
        ))}
    </Pie>
    <Tooltip />
    <Legend />
</PieChart>
</ResponsiveContainer>
</div>
</CardContent>
</Card>
</div>
```

末尾の `</div>` でグラフ用のグリッドが閉じます。ここを閉じ忘れると次のプロジェクト統計テーブルがグリッドの2列目に入り、円グラフの隣に半分の幅で置かれます。表の列が窮屈になったときはこの行の有無を先に確かめてください。

プロジェクト統計テーブルの見出し:

```
    { /* filepath: src/app/report/page.tsx(同じファイ  
    ルの続き) */}  
    { /* 完成版: プロジェクト統計テーブルの見出し */}  
    <Card>  
      <CardHeader>  
        <CardTitle>プロジェクト統計</CardTitle>  
      </CardHeader>  
      <CardContent>  
        <Table>  
          <TableHeader>  
            <TableRow>  
              <TableHead className="w-[200px]">  
                プロジェクト</TableHead>  
              <TableHead className="text-right">  
                タスク数</TableHead>  
              <TableHead className="text-right">  
                完了</TableHead>  
              <TableHead className="text-right">  
                進捗</TableHead>  
              <TableHead className="text-right">  
                作業時間</TableHead>  
            </TableRow>  
          </TableHeader>
```

Day 23: 週次レポートを表示しよう

Step 2 で読んだ見出しの定義はこの5列と同じ形です。あちらは読み比べ用なので貼ると同じ表が2つ並びます。表が二重になったときはこちらの1組だけを残してください。

プロジェクト統計テーブルの行:

```
    { /* filepath: src/app/report/page.tsx(同じ  
ファイルの続き) */}  
    { /* 完成版: プロジェクト統計テーブルの行 */}  
    <TableBody>  
      {overview?.projectStats.map((stat) => (  
        <TableRow key={stat.id}>  
          <TableCell className="font-medium">  
            {stat.name}</TableCell>  
          <TableCell className="text-right">  
            {stat.totalTasks}</TableCell>  
          <TableCell className="text-right">  
            {stat.completedTasks}</TableCell>  
          <TableCell className="text-right">  
            {stat.progress.toFixed(  
1)}}%</TableCell>  
          <TableCell className="text-right">  
            {stat.totalTimeHours.toFixed(1)}h  
          </TableCell>  
        </TableRow>  
      )})  
    </TableBody>
```

`projectStats` は `getOverview` が組み立てた配列で、Step 1 で確かめたとおり画面側では数え直しません。`toFixed(1)` は表示のときだけ小数を1桁へ丸める処理です。サーバーが返す `71.42857142857143` をそのまま出すと列の幅が行ごとに変わって表が読みにくくなります。

ファイル末尾の閉じタグ:

```

        { /* filepath: src/app/report/page.tsx(同じフ
ファイルの続き) */}

        { /* 完成版: 閉じタグと関数の終わり */}

        </Table>
      </CardContent>
    </Card>
  </div>
</AppLayout>
);
}

```

開いたタグを内側から順に閉じてファイルが終わります。今日の作業でリンクを足した位置はこの上のほうなので末尾の並びは Day 22 と変わりません。画面が真っ白になったときは増やした見出しの `div` の閉じタグが足りているかを先に数えてください。

今日のまとめ

- ☐ プロジェクト別統計を計算できました
- ☐ Table コンポーネントで一覧表示しました
- ☐ 週次レポートAPIを呼び出せました
- ☐ サマリーカードを表示しました

Day 23: 週次レポートを表示しよう

- ☐ 折れ線グラフで完了推移を表示しました
- ☐ 棒グラフで完了タスクの優先度別分布を表示しました

つまずきポイント

テーブルが空

原因

タスクやプロジェクトが0件か、`getOverview` の取得エラーが発生しているためです。

解決方法

データを追加し、開発者ツールの Network タブでエラーの有無を確認してください。

進捗率が NaN

原因

集計コードの写し間違いで `totalTasks > 0` の確認が抜けたためです。

解決方法

完成コードの `totalTasks > 0 ? ... : 0` と照合してください。

週次データが空

原因

初期データの完了タスクが2025年の日付で、担当者も別のユーザーのためです。

解決方法

自分が担当のタスクを1件「完了」にしてから読み込み直してください。

型エラーが出る

原因

weeks が string のためです。

解決方法

Number.parseInt で変換してください。

グラフが表示されない

原因

recharts がインストールされていないためです。

解決方法

Day 01 のセットアップ用スクリプトがインストールしています。プロジェクトのフォルダで `npm list recharts` を実行してください。(empty) と出たら、Day 22 Step 1 と同じ `npm install recharts@^3.2.1` を実行し、開発サーバーを起動し直してください。

今日学んだ用語

用語	意味
projectStats	プロジェクト別の集計結果配列
Table / TableRow	shadcn/ui のテーブル部品
getWeeklyReport	週次レポート取得 API

用語	意味
LineChart	Recharts の折れ線グラフ
BarChart	Recharts の棒グラフ
stackId	積み上げグラフにするための識別子

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `getWeeklyReport` の `where` はどんな行を集計から落としていますか。3つ挙げてください。

A. 1つ目は `completedAt` が `null` の行で、まだ終わっていないタスクです。2つ目は `assigneeId` が対象のユーザーと違う行で、他の人が片づけた仕事にあたります。3つ目は `startDate` より前に完了した行で、指定した週数の外になります。週次レポートは「この人がこの期間に終わらせた量」を出す画面なので残っているタスクの件数は扱いません。

Q2. `rangeEnd` を「明日の 00:00 UTC」ではなく「今この瞬間」にすると週平均カードの数字はどう変わりますか。

A. 開始日時も今から週数ぶん遡った時刻になるため4週間なら直近28日間を集計します。3週間に縮むことはありません。UTC の日付境界から現在と同じ時刻へ開始点が変わるので境界付近の完了タスクによって合計と平均が変わります。

Q3. 週の範囲判定を `>= weekStart && < weekEnd` と書き、終了側だけ「未満」にしているのはなぜですか。

A. 境界ちょうどに完了したタスクが前の週と次の週の両方で数えられるのを防ぐためです。「開始以上・終了未満」でそろえておけばどのタスクもどれか1つの週にだけ入ります。

追加課題: 2 週間の選択肢を追加する

短い期間だけを振り返れるようにします。サーバーが許している週数の範囲を使い、画面の選択肢を1つ増やしましょう。

前提は今日の週次レポートが表示できることです。

`src/app/report/weekly/page.tsx` の週数選択へ「2 週間」を追加してください。既存の「4 週間」の `SelectItem` を参考に、表示名と `value` を自分で決めます。サーバーの入力スキーマは1~12を許すので変更不要です。

「2 週間」を選び、グラフの週が2つになることを確認します。完了タスクが無くても2週間分の0件が表示されれば成功です。「4 週間」へ戻すと週が4つになることも確かめてください。

表示が変わらない場合は `SelectItem` の値と、`useQuery` に渡す `weeks` の変換を確認します。確認後は追加した選択肢を外し、「4 週間」へ戻します。週の境界で二重に数えない理由を、理解チェック Q3 から説明してみましょう。

次回予告

Day 24 では管理者専用のユーザー一覧ページを実装します。権限チェックでアクセスを制限し、ユーザー情報をテーブルで管理できるようにします。

次に読むもの

- 前の日: Day 22^{*1}
- 次の日: Day 24^{*2}
- 全体の地図: 学びのロードマップ^{*3}
- 目次: カリキュラム目次^{*4}
- 詰まったとき: トラブルシューティング^{*5}
- 言葉の意味: 用語集^{*6}

奥付

タイトル	Day 23: 週次レポートを表示しよう
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

-
1. https://drive.google.com/file/d/1kA8jhIcg7LJ3EGTWygNnABUihixSCD_C/view?usp=drivesdk
 2. <https://drive.google.com/file/d/1gHKuQHaL0tBpTM9qESjUqg3sHlqaKgPv/view?usp=drivesdk>
 3. https://drive.google.com/file/d/10py_p-6ry4RDlxBsyzPn4rtzZe74ND-T/view?usp=drivesdk
 4. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>
 5. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>
 6. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。